FINAL CHECKPOINT
ANALITIK DAN VISUALISASI DATA
STORE SALES FORECASTING DATASET

Disusun Oleh:

Muhammad Nadhir Sultan Azzaky

2509116080

KELAS B

PROGRAM STUDI SISTEM INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS MULAWARMAN

TAHUN 2026

DAFTAR ISI

DAFTAR ISI ii
DAFTAR GAMBAR iii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan Analisis 1
BAB II PEMBAHASAN 3
2.1 Business Understanding 3
2.1.1 Business Objective 3
2.1.2 Asess Situation 3
2.1.3 Analytic Goals 4
2.1.4 Project Plan 4
2.2 Data Understanding & EDA 5
2.2.1 Data Understanding 5
2.2.2 EDA 30
2.3 Data Preparation 37
2.3.1 Penanganan Tipe Data 37
2.3.2 Incosistent Values 39
2.3.3 Missing Values 40
2.3.4 Duplicated Values 41
2.3.5 Outliers 41
2.3.6 Construct Data 43
2.3.7 Data Reduction 45
BAB III PENUTUP 47
3.1 Kesimpulan 47
3.2 Saran 47

DAFTAR GAMBAR

Gambar 1 df..... 6
Gambar 2 df.info() 6
Gambar 3 df.shape 8
Gambar 4 df.describe()..... 9
Gambar 5 df.dypes 11
Gambar 6 Kolom Row ID 12
Gambar 7 Kolom Order ID 13
Gambar 8 Kolom Order Date 13
Gambar 9 Kolom Ship Date 14
Gambar 10 Kolom Ship Mode 15
Gambar 11 Kolom Customer ID 15
Gambar 12 Kolom Customer Name 16
Gambar 13 Kolom Segment 17
Gambar 14 Kolom Country 17
Gambar 15 Kolom City 18
Gambar 16 Kolom State 19
Gambar 17 Kolom Postal Code 20
Gambar 18 Kolom Region 20
Gambar 19 Kolom Product ID 21
Gambar 20 Kolom Category 22
Gambar 21 Kolom Sub-Category 22
Gambar 22 Kolom Product Name 23
Gambar 23 Kolom Sales 24
Gambar 24 Kolom Quantity 24
Gambar 25 Kolom Discount 25
Gambar 26 Kolom Profit 25
Gambar 27 Missing Values 26
Gambar 28 Duplicated Values 27
Gambar 29 Outliers Values 29
Gambar 30 Comparison 31
Gambar 31 Composition 33
Gambar 32 Distribution 34
Gambar 33 Relationship 36
Gambar 34 df.info() before 37
Gambar 35 df.info() after 38

Gambar 36 Missing Values 40
Gambar 37 Construct Data..... 45
Gambar 38 Data Reduction..... 46

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi di era digital saat ini membuat data menjadi salah satu sumber informasi yang sangat penting bagi perusahaan. Data dapat dimanfaatkan untuk memahami kondisi bisnis, melihat pola penjualan, serta membantu dalam menentukan strategi yang tepat. Dengan jumlah data yang semakin besar, perusahaan membutuhkan cara yang efektif untuk mengolah dan memahami data tersebut agar dapat digunakan sebagai dasar dalam pengambilan keputusan.

Namun, data yang tersedia sering kali masih berupa data mentah sehingga sulit dipahami apabila hanya disajikan dalam bentuk tabel. Hal ini dapat menyulitkan dalam mengidentifikasi pola, tren, serta hubungan antar data. Oleh karena itu, diperlukan proses analisis dan visualisasi data agar informasi yang terdapat dalam data dapat disajikan dengan lebih jelas dan mudah dipahami.

Dalam analisis ini digunakan **Store Sales Forecasting Dataset** yang berisi data transaksi penjualan sebuah toko, seperti tanggal pesanan, wilayah penjualan, kategori produk, jumlah penjualan, diskon, serta profit. Melalui proses analisis dan visualisasi data menggunakan Tableau, diharapkan dapat diperoleh berbagai insight terkait performa penjualan serta faktor-faktor yang mempengaruhinya. Hasil analisis tersebut kemudian disajikan dalam bentuk dashboard interaktif agar informasi yang dihasilkan dapat lebih mudah dipahami dan digunakan dalam mendukung pengambilan keputusan.

1.2 Tujuan Analisis

Adapun tujuan dari analisis data yang dilakukan dalam laporan ini adalah sebagai berikut:

1. Menganalisis data penjualan pada **Store Sales Forecasting Dataset** untuk memahami kondisi penjualan yang terjadi.
2. Mengidentifikasi pola dan tren penjualan berdasarkan kategori produk, wilayah penjualan, dan periode waktu tertentu.
3. Menganalisis hubungan antara diskon yang diberikan dengan profit yang diperoleh dari setiap transaksi.
4. Menyajikan hasil analisis dalam bentuk visualisasi data yang informatif dan mudah dipahami.
5. Membuat dashboard interaktif menggunakan Tableau untuk membantu dalam memahami data serta mendukung proses pengambilan keputusan.

BAB II
PEMBAHASAN

2.1 Business Understanding

Business Understanding merupakan tahap awal dalam proses analisis data yang bertujuan untuk memahami permasalahan bisnis serta tujuan yang ingin dicapai melalui analisis data. Pada tahap ini dilakukan pemahaman terhadap konteks bisnis, kondisi yang sedang terjadi, serta kebutuhan informasi yang diperlukan untuk mendukung pengambilan keputusan. Selain itu, tahap ini juga mencakup identifikasi tujuan bisnis, penentuan tujuan analisis, serta perencanaan langkah-langkah yang akan dilakukan dalam proses analisis data sehingga proses analisis dapat dilakukan secara lebih terarah dan sesuai dengan kebutuhan yang ingin diselesaikan. Tahapan-tahapan dalam *Business Understanding* tersebut dapat dijelaskan sebagai berikut:

2.1.1 Business Objective

Dataset yang digunakan pada analisis ini berisi data penjualan dari beberapa toko yang mencakup informasi seperti tanggal transaksi, produk, promo dan jumlah barang yang terjual. Melalui data ini, analisis dilakukan untuk mengetahui pola penjualan serta faktor faktor yang mempengaruhi naik turunnya sales sehingga hasilnya bisa membantu perusahaan dalam menyusun strategi promosi, mengatur persediaan barang dan merencanakan target penjualan di periode berikutnya.

2.1.2 Asess Situation

Walaupun perusahaan sudah punya data penjualan dalam jumlah besar. Informasi tersebut belum dimanfaatkan dengan baik hingga masih kesulitan untuk mengetahui waktu penjualan paling tinggi, dampak promo terhadap sales, cara mengatur stock dan memperkirakan penjualan selanjutnya, oleh

karena itu diperlukan analisis data untuk mendapatkan insight yang bisa membantu pengambilan keputusan.

2.1.3 Analytic Goals

Tujuan analisis data yang ingin dicapai dari proyek ini adalah:

1. Mengetahui perkembangan penjualan dari waktu ke waktu.
2. Mengetahui pengaruh promo terhadap jumlah penjualan.
3. Menyiapkan dasar informasi untuk prediksi penjualan di masa depan.

2.1.4 Project Plan

Untuk mencapai tujuan tersebut. Langkah langkah yang akan dilakukan yaitu memuat dataset ke dalam Google Colab, memahami isi dan struktur data, lalu melakukan pembersihan jika terdapat data yang kosong/tidak sesuai, lalu melakukan eksplorasi untuk menemukan pola dan yang terakhir menarik insight dari hasil analisis yang telah dilakukan.

2.2 Data Understanding & EDA

Data Understanding merupakan tahapan dalam proses analisis data yang bertujuan untuk memahami dataset yang digunakan dalam penelitian. Pada tahap ini dilakukan peninjauan terhadap struktur data, jenis data, jumlah data, serta informasi yang terdapat pada setiap variabel dalam dataset. Selain itu, dilakukan juga Exploratory Data Analysis (EDA) untuk memperoleh gambaran awal mengenai kondisi data dan menemukan kemungkinan permasalahan yang ada, seperti ketidaksesuaian tipe data, *missing values*, data duplikat, nilai yang tidak konsisten, maupun *outliers*. Tahapan yang dilakukan dalam proses ini dapat dijelaskan sebagai berikut:

2.2.1 Data Understanding

Dalam *Understanding*, ada beberapa *library Python* yang diperlukan untuk membantu membaca *dataset*, yaitu:

1. Import Library

```
import pandas as pd
```

Perintah import pandas as pd ini berfungsi untuk membaca *dataset*.

2. Memuat Dataset

```
file = '/content/drive/MyDrive/CP1 AVD/stores_sales_forecasting.csv'
df = pd.read_csv(file, encoding='latin1')
df
```

Perintah diatas ini berfungsi untuk menampilkan *dataset.*

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Sales	Quantity	Discount	Profit
0	1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	42420	South	FUR-BO-10001798	Furniture	Bookcases	Bush Somerset Collection Bookcase	261.9600	2	0.00	41.9136
1	2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	42420	South	FUR-CH-10000454	Furniture	Chairs	Hon Deluxe Fabric Upholstered Stacking Chairs,...	731.9400	3	0.00	219.5820
2	4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	33311	South	FUR-TA-10000577	Furniture	Tables	Bretford CR4500 Series Slim Rectangular Table	957.5775	5	0.45	-383.0310
3	6	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	...	90032	West	FUR-FU-10001487	Furniture	Furnishings	Eldon Expressions Wood and Plastic Desk Access...	48.8600	7	0.00	14.1694
4	11	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	...	90032	West	FUR-TA-10001539	Furniture	Tables	Chromcraft Rectangular Conference Tables	1706.1840	9	0.20	85.3092
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2116	9963	CA-2015-168088	3/19/2015	3/22/2015	First Class	CM-12655	Corinna Mitchell	Home Office	United States	Houston	...	77041	Central	FUR-BO-10004218	Furniture	Bookcases	Bush Heritage Pine Collection 5-Shelf Bookcase...	383.4656	4	0.32	-67.6704
2117	9965	CA-2015-146374	12/5/2016	12/10/2016	Second Class	HE-14800	Harold Engle	Corporate	United States	Newark	...	19711	East	FUR-FU-10002671	Furniture	Furnishings	Electrix 20W Halogen Replacement Bulb for Zoom...	13.4000	1	0.00	6.4320
2118	9981	US-2015-151435	9/6/2015	9/9/2015	Second Class	SW-20455	Shaun Weien	Consumer	United States	Lafayette	...	70506	South	FUR-TA-10001039	Furniture	Tables	KI Adjustable-Height Table	85.9800	1	0.00	22.3548
2119	9990	CA-2014-110422	1/21/2014	1/23/2014	Second Class	TB-21400	Tom Boeckenhauer	Consumer	United States	Miami	...	33180	South	FUR-FU-10001889	Furniture	Furnishings	Ultra Door Pull Handle	25.2480	3	0.20	4.1028
2120	9991	CA-2017-121258	2/26/2017	3/3/2017	Standard Class	DB-13060	Dave Brooks	Consumer	United States	Costa Mesa	...	92627	West	FUR-FU-10000747	Furniture	Furnishings	Tenex B1-RE Series Chair Mats for Low Pile Car...	91.9600	2	0.00	15.6332

2121 rows × 21 columns

Gambar 1 df

3. Struktur Data

```
df.info()
```

Perintah df.info() berfungsi untuk menampilkan ringkasan terstruktur mengenai dataset.

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2121 entries, 0 to 2120
Data columns (total 21 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Row ID         2121 non-null   int64
 1   Order ID       2121 non-null   object
 2   Order Date     2121 non-null   object
 3   Ship Date      2121 non-null   object
 4   Ship Mode      2121 non-null   object
 5   Customer ID    2121 non-null   object
 6   Customer Name  2121 non-null   object
 7   Segment        2121 non-null   object
 8   Country        2121 non-null   object
 9   City           2121 non-null   object
 10  State          2121 non-null   object
 11  Postal Code    2121 non-null   int64
 12  Region         2121 non-null   object
 13  Product ID     2121 non-null   object
 14  Category       2121 non-null   object
 15  Sub-Category   2121 non-null   object
 16  Product Name   2121 non-null   object
 17  Sales          2121 non-null   float64
 18  Quantity       2121 non-null   int64
 19  Discount       2121 non-null   float64
 20  Profit         2121 non-null   float64
dtypes: float64(3), int64(3), object(15)
memory usage: 348.1+ KB
```

Gambar 2 df.info()

Berdasarkan hasil eksekusi tersebut, dapat diketahui bahwa dataset memiliki total 2.121 baris data dan 21 kolom. *Output* ini juga mengonfirmasi bahwa tidak terdapat data yang kosong atau *missing*

values, karena seluruh kolom memiliki jumlah data terisi *non-null* yang utuh sebanyak 2.121.

a) Row ID: Jumlah baris: 2121 Tipe data: int64 Deskripsi: Nomor identitas atau urutan unik untuk setiap baris data.
b) Order ID: Jumlah baris: 2121 Tipe data: object Deskripsi: Nomor identitas unik untuk setiap pesanan.
c) Order Date: Jumlah baris: 2121 Tipe data: object Deskripsi: Tanggal saat transaksi pesanan dilakukan.
d) Ship Date: Jumlah baris: 2121 Tipe data: object Deskripsi: Tanggal saat pesanan dikirimkan ke pelanggan.
e) Ship Mode: Jumlah baris: 2121 Tipe data: object Deskripsi: Metode atau jalur pengiriman pesanan.
f) Customer ID: Jumlah baris: 2121 Tipe data: object Deskripsi: Nomor identitas unik untuk setiap pelanggan.
g) Customer Name: Jumlah baris: 2121 Tipe data: object Deskripsi: Nama lengkap pelanggan pembeli.
h) Segment: Jumlah baris: 2121 Tipe data: object Deskripsi: Segmen atau kategori pelanggan.
i) Country: Jumlah baris: 2121 Tipe data: object Deskripsi: Negara tujuan pengiriman pesanan.
j) City: Jumlah baris: 2121 Tipe data: object Deskripsi: Kota tujuan pengiriman pesanan.
k) State: Jumlah baris: 2121 Tipe data: object Deskripsi: Negara bagian atau provinsi tujuan pengiriman.
l) Postal Code: Jumlah baris: 2121 Tipe data: int64 Deskripsi: Kode pos tujuan pengiriman pesanan.
m) Region: Jumlah baris: 2121 Tipe data: object Deskripsi: Wilayah atau kawasan regional pelanggan.
n) Product ID: Jumlah baris: 2121 Tipe data: object Deskripsi: Nomor identitas unik untuk setiap produk.

o) Category: Jumlah baris: 2121 Tipe data: object Deskripsi: Kategori utama dari produk yang dipesan.
p) Sub-Category: Jumlah baris: 2121 Tipe data: object Deskripsi: Sub-kategori atau spesifikasi jenis produk.
q) Product Name: Jumlah baris: 2121 Tipe data: object Deskripsi: Nama lengkap atau judul produk.
r) Sales: Jumlah baris: 2121 Tipe data: float64 Deskripsi: Total nilai pendapatan atau nominal penjualan.
s) Quantity: Jumlah baris: 2121 Tipe data: int64 Deskripsi: Jumlah unit produk yang dibeli dalam satu pesanan.
t) Discount: Jumlah baris: 2121 Tipe data: float64 Deskripsi: Nilai potongan harga yang diberikan pada transaksi.
u) Profit: Jumlah baris: 2121 Tipe data: float64 Deskripsi: Nominal keuntungan bersih yang didapatkan dari penjualan.

```
df.shape()
```

Perintah df.shape digunakan untuk mengetahui dimensi atau ukuran dari *DataFrame*.

```
df.shape
(2121, 21)
```

Gambar 3 df.shape

Berdasarkan hasil eksekusi tersebut, terkonfirmasi bahwa dataset yang digunakan memiliki ukuran total 2.121 baris dan 21 kolom.

4. Statistik Deskriptif

```
df.describe(include='all')
```

Perintah df.describe(include='all') berfungsi untuk menampilkan ringkasan statistik deskriptif secara menyeluruh untuk semua kolom, baik yang bertipe angka (numerik) maupun teks (kategorikal).

```
df.describe(include='all')
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Sales	Quantity	Discount	Profit
count	2121.000000	2121	2121	2121	2121	2121	2121	2121	2121	2121	...	2121.000000	2121	2121	2121	2121	2121	2121.000000	2121.000000	2121.000000	2121.000000
unique	NaN	1764	889	960	4	707	707	3	1	371	...	NaN	4	375	1	4	380	NaN	NaN	NaN	NaN
top	NaN	US-2015-138121	9/5/2016	12/6/2017	Standard Class	SV-20365	Seth Vernon	Consumer	United States	New York City	...	NaN	West	FUR-FU-10004270	Furniture	Furnishings	Ki Adjustable-Height Table	NaN	NaN	NaN	NaN
freq	NaN	4	10	10	1248	15	15	1113	2121	192	...	NaN	707	16	2121	957	18	NaN	NaN	NaN	NaN
mean	5041.643564	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	55726.556341	NaN	NaN	NaN	NaN	NaN	349.834887	3.785007	0.173923	8.699327
std	2885.740258	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	32261.888225	NaN	NaN	NaN	NaN	NaN	503.179145	2.251620	0.181547	136.049246
min	1.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	1040.000000	NaN	NaN	NaN	NaN	NaN	1.892000	1.000000	0.000000	-1862.312400
25%	2568.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	22801.000000	NaN	NaN	NaN	NaN	NaN	47.040000	2.000000	0.000000	-12.849000
50%	5145.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	60505.000000	NaN	NaN	NaN	NaN	NaN	182.220000	3.000000	0.200000	7.774800
75%	7534.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	90032.000000	NaN	NaN	NaN	NaN	NaN	435.168000	5.000000	0.300000	33.726600
max	9991.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	99301.000000	NaN	NaN	NaN	NaN	NaN	4416.174000	14.000000	0.700000	1013.127000

11 rows × 21 columns

Gambar 4 df.describe()

Adapun insight mengenai kondisi dan kelengkapan data pada dataset yang didapat pada informasi diatas ini, yaitu:

1. Count

   Berdasarkan data, total baris dataset adalah 2.121 data. Maka dapat disimpulkan: Seluruh kolom (dari Row ID hingga Profit) memiliki jumlah data sebanyak 2.121.

   Kesimpulan: Tidak ada kolom yang mengandung *missing values*.

2. Mean vs Median (50%)

   a. Kolom Sales

- Mean = 349.83
- Median = 182.22

Nilai rata-rata jauh lebih besar dari median, menunjukkan adanya beberapa transaksi dengan nilai penjualan sangat tinggi yang menarik rata-rata ke atas (*outlier*).

b. Kolom Quantity

- Mean = 3.78
- Median = 3.00

Nilai rata-rata sedikit lebih besar dari median, menunjukkan distribusi jumlah barang per pesanan cukup merata namun dengan sedikit kecondongan pada pesanan berjumlah besar.

c. Kolom Profit

- Mean = 8.69
- Median = 7.77

Nilai rata-rata sedikit lebih besar dari median, namun rentang data sangat dipengaruhi oleh nilai kerugian dan keuntungan ekstrem.

Kesimpulan: Secara keseluruhan, distribusi data finansial (terutama pada *Sales*) bersifat *skewed right* (condong ke kanan). Terdapat beberapa nilai transaksi yang ekstrem sehingga memengaruhi nilai rata-rata keseluruhan.

3. Min dan Max

a. Sales

- Min = 1.89
- Max = 4416.17

b. Quantity

- Min = 1
- Max = 14

c. Profit

- Min = -1862.31
- Max = 1013.12

Kesimpulan: Terdapat nilai minimum negatif yang sangat besar pada *Profit* (-1862.31) yang perlu diperhatikan karena menandakan adanya transaksi yang sangat merugikan. Nilai *Sales* maksimum mencapai 4416.17 menunjukkan adanya pesanan grosir atau produk premium bernilai sangat tinggi dibandingkan pesanan terkecil.

4. Standar Deviasi (std) - Variabilitas Data

a. Sales

- Mean = 349.83
- Std = 503.17

b. Profit

- Mean = 8.69
- Std = 136.04

Kesimpulan: Pada kolom *Sales*, standar deviasi yang lebih besar dari rata-rata (mean) menunjukkan variasi nominal transaksi sangat beragam. Pada kolom *Profit*, nilai standar deviasi yang sangat tinggi menandakan tingkat margin keuntungan per pesanan sangat tidak stabil, dengan sebaran yang luas antara transaksi yang untung besar dan rugi besar.

5. Cek Tipe Data Pada Setiap Kolom

```
df.dftypes
```

Perintah df.dtypes berfungsi untuk mengidentifikasi dan menampilkan rincian tipe data pada masing-masing kolom di dalam dataset.

```
df.dtypes
```

	0
Row ID	int64
Order ID	object
Order Date	object
Ship Date	object
Ship Mode	object
Customer ID	object
Customer Name	object
Segment	object
Country	object
City	object
State	object
Postal Code	int64
Region	object
Product ID	object
Category	object
Sub-Category	object
Product Name	object
Sales	float64
Quantity	int64
Discount	float64
Profit	float64

dtype: object

Gambar 5 df.dypes

Berdasarkan informasi tersebut, struktur data dalam tabel ini terbagi menjadi tiga jenis tipe data:

1. int64: Terdapat pada 3 kolom, yaitu Row ID, Postal Code, dan Quantity.
2. float64: Terdapat pada 3 kolom, yaitu Sales, Discount, dan Profit.
3. object: Mendominasi dataset pada 15 kolom lainnya, yang mencakup data identitas (seperti Order ID dan Customer ID), detail wilayah, hingga nama produk. Sebagai catatan, kolom yang berisi informasi tanggal seperti Order Date dan Ship Date saat ini masih terbaca sebagai *object* dan belum menjadi *datetime*.

6. Incosistent Values

Inconsistent Values adalah kondisi di mana data yang merujuk pada informasi atau makna yang sama ditulis dengan format, ejaan, atau standar yang berbeda di dalam sebuah *dataset*.

a. Row ID

```
print(df['Row ID'].unique())
```

Perintah print(df['Row ID'].unique()) berfungsi untuk mengidentifikasi dan menampilkan seluruh nilai unik (berbeda) yang terdapat di dalam kolom Row ID.

```
print(df['Row ID'].unique())

[   1    2    4 ... 9981 9990 9991]
```

Gambar 6 Kolom Row ID

Berdasarkan hasil keluaran tersebut, terdapat sebuah temuan penting, meskipun total data sebelumnya diketahui hanya berjumlah 2.121 baris, nilai Row ID justru mencapai angka 9991 dan menunjukkan adanya nomor urut yang terlampaui atau melompat (misalnya dari angka 2 langsung ke 4).

Hal ini mengindikasikan bahwa *dataset* yang sedang dianalisis saat ini kemungkinan besar merupakan hasil *filter* dari sebuah *dataset* sumber yang ukurannya jauh lebih besar.

b. Order ID

```
print(df['Order ID'].unique())
```

Perintah print(df['Order ID'].unique()) berfungsi untuk mengidentifikasi dan menampilkan seluruh nilai unik atau daftar nomor pesanan yang berbeda di dalam kolom Order ID.

```
print(df['Order ID'].unique())

['CA-2016-152156' 'US-2015-108966' 'CA-2014-115812' ... 'US-2015-151435'
 'CA-2014-110422' 'CA-2017-121258']
```

Gambar 7 Kolom Order ID

Berdasarkan hasil keluaran tersebut, dapat diidentifikasi bahwa data nomor pesanan memiliki pola penulisan atau format yang konsisten dan terstruktur.

c. Order Date

```
print(df['Order Date'].unique())
```

Perintah print(df['Order Date'].unique()) berfungsi untuk menampilkan seluruh nilai unik atau tanggal transaksi yang berbeda pada kolom Order Date.

```
print(df['Order Date'].unique())

... ['11/8/2016' '10/11/2015' '6/9/2014' '7/16/2017' '9/25/2015' '9/17/2015'
 '12/8/2016' '12/27/2015' '4/18/2015' '6/17/2016' '11/24/2015' '4/30/2015'
 '4/26/2015' '12/9/2017' '11/26/2014' '5/28/2017' '1/31/2015' '11/9/2017'
 '11/13/2015' '12/25/2017' '3/2/2015' '6/12/2016' '12/26/2014' '9/20/2014'
 '11/6/2016' '10/13/2016' '9/5/2016' '10/22/2014' '12/5/2016' '3/1/2014'
 '9/8/2014' '8/5/2014' '11/21/2015' '10/12/2015' '8/3/2014' '6/3/2017'
 '1/2/2015' '12/24/2015' '8/9/2015' '2/28/2015' '4/7/2017' '6/4/2016'
 '6/1/2014' '12/10/2016' '11/28/2015' '7/30/2015' '11/2/2015' '11/20/2016'
 '10/28/2016' '11/19/2017' '5/4/2015' '9/12/2016' '9/27/2014' '11/4/2014'
 '11/3/2017' '8/30/2016' '7/12/2014' '4/8/2016' '9/25/2017' '10/21/2016'
 '8/27/2017' '11/27/2015' '12/3/2015' '9/8/2016' '12/8/2017' '4/14/2016'
 '10/19/2017' '8/21/2017' '11/23/2017' '1/20/2017' '3/20/2017'
 '10/20/2017' '2/12/2014' '9/26/2016' '4/22/2016' '1/17/2015' '3/31/2017'
 '11/9/2014' '10/27/2016' '6/10/2017' '3/18/2016' '7/25/2016' '5/30/2016'
 '11/26/2017' '1/23/2017' '10/21/2017' '11/7/2015' '9/7/2017' '12/7/2015'
 '5/11/2014' '4/15/2016' '9/11/2017' '6/10/2016' '11/20/2017' '9/14/2014'
 '10/1/2017' '7/20/2017' '7/3/2015' '8/8/2014' '3/15/2014' '8/18/2017'
 '11/29/2015' '9/15/2017' '10/4/2015' '5/20/2016' '12/18/2016'
 '12/11/2016' '9/19/2016' '8/24/2015' '12/24/2014' '6/9/2017' '3/18/2017'
 '6/21/2014' '9/2/2017' '11/4/2016' '4/9/2015' '12/2/2014' '1/11/2014'
 '9/24/2017' '6/2/2014' '12/15/2016' '10/7/2014' '12/3/2016' '1/21/2017'
 '5/27/2014' '10/2/2017' '9/18/2016' '5/14/2014' '5/19/2016' '1/30/2017'
 '8/21/2015' '10/3/2015' '12/12/2015' '6/26/2015' '2/20/2017' '4/22/2017'
 '2/3/2015' '10/13/2014' '5/14/2017' '6/28/2014' '6/16/2017' '11/20/2015'
 '10/13/2015' '1/1/2017' '2/19/2016' '1/10/2014' '1/17/2016' '11/30/2017'
 '10/22/2016' '4/29/2014' '12/30/2017' '5/31/2015' '11/27/2017'
 '6/15/2014' '12/13/2016' '7/26/2015' '11/28/2017' '4/3/2017' '12/28/2017'
 '9/22/2015' '11/14/2017' '1/16/2014' '10/3/2016' '11/11/2014' '5/21/2017'
 '3/29/2015' '10/28/2015' '7/31/2015' '11/24/2014' '6/17/2017'
 '12/21/2015' '7/6/2015' '4/27/2015' '7/11/2014' '3/20/2016' '9/6/2016'
 '10/12/2017' '3/8/2016' '4/4/2015' '3/23/2015' '10/1/2016' '3/1/2016'
 '6/4/2017' '6/25/2014' '6/14/2016' '11/19/2016' '12/18/2017' '1/20/2014'
 '4/12/2016' '9/29/2014' '5/8/2015' '12/20/2014' '7/2/2016' '12/1/2016'
 '2/26/2017' '9/15/2015' '7/18/2016' '7/14/2016' '9/9/2017' '12/4/2016'
```

Gambar 8 Kolom Order Date

Berdasarkan hasil keluaran tersebut, terlihat variasi tanggal pesanan yang mencakup rentang waktu beberapa tahun. Temuan penting dari *output* ini adalah nilai tanggal masih direpresentasikan sebagai teks atau *string* dengan format Bulan/Hari/Tahun. Hal ini mengindikasikan perlunya proses transformasi data atau *data cleaning,* untuk mengubah tipe data kolom ini dari *object* menjadi *datetime* agar analisis tren waktu dapat dilakukan dengan akurat.

d. Ship Date

```
print(df['Ship Date'].unique())
```

Perintah print(df['Ship Date'].unique()) berfungsi untuk menampilkan seluruh nilai unik atau daftar tanggal pengiriman yang berbeda pada kolom Ship Date.

Gambar 9 Kolom Ship Date

Berdasarkan hasil keluaran tersebut, terlihat variasi tanggal yang mencakup rentang waktu dari tahun 2014 hingga awal 2018. Serupa dengan temuan pada kolom *Order Date*, data tanggal pada kolom ini juga masih terekam sebagai *string* dengan format Bulan/Hari/Tahun. Oleh karena itu, kolom ini juga memerlukan konversi tipe data menjadi *datetime* agar nantinya dapat diolah untuk analisis lanjutan.

e. Ship Mode

```
print(df['Ship Mode'].unique())
```

Perintah print(df['Ship Mode'].unique()) berfungsi untuk mengidentifikasi dan menampilkan seluruh nilai unik atau pilihan metode pengiriman yang terdapat di dalam kolom Ship Mode.

```
print(df['Ship Mode'].unique())

['Second Class' 'Standard Class' 'First Class' 'Same Day']
```

Gambar 10 Kolom Ship Mode

Berdasarkan hasil keluaran tersebut, dapat diketahui bahwa hanya terdapat 4 kategori pengiriman yang ditawarkan, yaitu *Second Class*, *Standard Class*, *First Class*, dan *Same Day*. Temuan ini juga menunjukkan bahwa penulisan data pada kolom ini sudah sangat rapi dan konsisten, karena tidak ditemukan adanya *inconsistent values*.

f. Customer ID

```
print(df['Customer ID'].unique())
```

Perintah print(df['Customer ID'].unique()) berfungsi untuk menampilkan daftar lengkap nilai unik atau identitas pelanggan yang berbeda di dalam kolom Customer ID.

```
print(df['Customer ID'].unique())

['CG-12520' 'SO-20335' 'BH-11710' 'SF-20065' 'EB-13870' 'TB-21520'
 'GH-14485' 'SN-20710' 'DP-13000' 'TB-21055' 'KM-16720' 'PS-18970'
 'JE-15745' 'KB-16600' 'PO-18865' 'KB-16315' 'PN-18775' 'LC-16870'
 'CV-12805' 'DK-13090' 'SC-20725' 'AD-10180' 'PF-19165' 'LS-16975'
 'LC-16885' 'JD-15895' 'MG-17680' 'JE-16165' 'DB-13060' 'BM-11140'
 'CS-12130' 'MP-17470' 'BS-11755' 'HK-14890' 'VW-21775' 'JB-15925'
 'DS-13180' 'VD-21670' 'CS-12400' 'KL-16645' 'DW-13480' 'LH-17155'
 'DR-12880' 'HA-14920' 'JO-15280' 'NZ-18565' 'KL-16555' 'JH-15910'
 'AS-10225' 'BP-11185' 'TS-21205' 'NM-18445' 'DJ-13510' 'PO-18850'
 'AC-10420' 'JL-15505' 'KW-16435' 'MK-17905' 'AR-10405' 'DO-13435'
 'DK-13225' 'AG-10675' 'KH-16510' 'KC-16675' 'PB-19150' 'MP-17965'
 'NF-18385' 'JM-15265' 'PK-19075' 'TB-21595' 'EB-13705' 'SC-20095'
 'TN-21040' 'JS-15940' 'JP-15520' 'JE-15475' 'MT-18070' 'RO-19780'
 'MY-17380' 'LS-17245' 'MC-17605' 'PH-18790' 'KB-16240' 'JC-15340'
 'DK-12985' 'PG-18895' 'BD-11320' 'RC-19960' 'OT-18730' 'LP-17080'
 'CA-12775' 'EB-13840' 'CS-11950' 'BD-11725' 'LR-16915' 'AS-10240'
 'NG-18355' 'SF-20965' 'MO-17800' 'DJ-13420' 'RB-19465' 'JD-16150'
 'AR-10825' 'EH-14125' 'CB-12025' 'RP-19390' 'CD-11980' 'RA-19285'
 'NP-18325' 'DI-13600' 'AS-10285' 'JK-15370' 'DP-13165' 'SV-20365'
 'KC-16540' 'CK-12325' 'MR-17545' 'EG-13900' 'DS-13030' 'MH-17785'
 'RF-19735' 'MS-17830' 'NB-18655' 'AJ-10795' 'LE-16810' 'BV-11245'
 'BM-11785' 'LT-17110' 'SA-20830' 'CD-11920' 'HW-14935' 'GM-14440'
 'AA-10375' 'ZC-21910' 'PJ-19015' 'BM-11650' 'AF-10885' 'GA-14725'
 'GT-14710' 'AJ-10780' 'CC-12430' 'HP-14815' 'PM-19135' 'LL-16840'
 'SC-20050' 'RW-19630' 'RB-19795' 'KH-16360' 'GH-14665' 'DL-12925'
 'LW-16990' 'DL-13315' 'TZ-21580' 'JG-15805' 'SL-20155' 'RD-19585'
 'RH-19495' 'VG-21790' 'JS-15880' 'KM-16225' 'LA-16780' 'EL-13735'
 'RD-19660' 'MO-17500' 'DG-13300' 'NF-18595' 'JD-16060' 'BD-11620'
 'BT-11395' 'KD-16270' 'NS-18640' 'PV-18985' 'EH-13945' 'RA-19915'
 'HL-15040' 'BD-11605' 'AP-10915' 'CB-12535' 'PS-19045' 'GB-14530'
 'BT-11530' 'LS-16945' 'SW-20755' 'SP-20620' 'PG-18820' 'GM-14680'
 'NP-18700' 'DJ-13630' 'MH-18115' 'CC-12220' 'JC-16105' 'BG-11695'
 'MS-17710' 'TB-21355' 'LC-17140' 'TC-21535' 'CS-12460' 'RC-19825'
 'HG-14965' 'LD-17005' 'JF-15190' 'CM-12115' 'BP-11290' 'LB-16735'
 'KT-16465' 'AB-10600' 'SZ-20035' 'JR-15670' 'CA-12265' 'JK-16120'
```

Gambar 11 Kolom Customer ID

Berdasarkan hasil keluaran tersebut, dapat diidentifikasi bahwa data ID pelanggan memiliki pola penulisan yang sangat konsisten dan terstruktur. Setiap ID dibentuk dari gabungan dua huruf kapital di awal, diikuti oleh tanda hubung (-), dan diakhiri dengan kombinasi lima digit angka. Temuan ini mengindikasikan bahwa data pada kolom ini sudah rapi dan terbebas dari masalah *inconsistent values* terkait format penulisan.

g. Customer Name

```
print(df['Customer Name'].unique())
```

Perintah print(df['Customer Name'].unique()) berfungsi untuk menampilkan daftar lengkap seluruh nama pelanggan yang unik atau berbeda pada kolom Customer Name.

```
print(df['Customer Name'].unique())

['Claire Gute' "Sean O'Donnell" 'Brosina Hoffman' 'Sandra Flanagan'
 'Emily Burns' 'Tracy Blumstein' 'Gene Hale' 'Steve Nguyen'
 'Darren Powers' 'Ted Butterfield' 'Kunst Miller' 'Paul Stevenson'
 'Joel Eaton' 'Ken Brennan' "Patrick O'Donnell" 'Karl Braun'
 'Parhena Norris' 'Lena Cacioppo' 'Cynthia Voltz' 'Dave Kipp'
 'Steven Cartwright' 'Alan Dominguez' 'Philip Fox' 'Lindsay Shagiari'
 'Lena Creighton' 'Jonathan Doherty' 'Maureen Gastineau' 'Justin Ellison'
 'Dave Brooks' 'Becky Martin' 'Chad Sievert' 'Mark Packer' 'Bruce Stewart'
 'Heather Kirkland' 'Victoria Wilson' 'Joni Blumstein' 'David Smith'
 'Valerie Dominguez' 'Christopher Schild' 'Ken Lonsdale' 'Dianna Wilson'
 'Logan Haushalter' 'Dan Reichenbach' 'Helen Andreada' "Jas O'Carroll"
 'Nick Zandusky' 'Kelly Lampkin' 'Jonathan Howell' 'Alan Schoenberger'
 'Ben Peterman' 'Thomas Seio' 'Nathan Mautz' 'Don Jones' "Patrick O'Brill"
 'Alyssa Crouse' 'Jeremy Lonsdale' 'Katrina Willman' 'Michael Kennedy'
 'Allen Rosenblatt' 'Denny Ordway' 'Dean Katz' 'Anna Gayman'
 'Keith Herrera' 'Kimberly Carter' 'Philip Brown' 'Michael Paige'
 'Natalie Fritzler' 'Janet Molinari' 'Pete Kriz' 'Troy Blackwell'
 'Ed Braxton' 'Sanjit Chand' 'Tanja Norvell' 'Joni Sundaresam'
 'Jeremy Pistek' 'Jeremy Ellison' 'Michelle Tran' "Rose O'Brian"
 'Maribeth Yedwab' 'Lynn Smith' 'Matt Connell' 'Patricia Hirasaki'
 'Karen Bern' 'Jasper Cacioppo' 'Darren Koutras' 'Paul Gonzalez'
 'Bill Donatelli' 'Ryan Crowe' 'Olvera Toch' 'Liz Pelletier'
 'Cynthia Arntzen' 'Ellis Ballard' 'Carlos Soltero' 'Bruce Degenhardt'
 'Lena Radford' 'Alan Shonely' 'Nat Gilpin' 'Sylvia Foulston'
 "Meg O'Connel" 'Denny Joy' 'Rick Bensley' 'Justin Deggeller'
 'Anthony Rawles' 'Eugene Hildebrand' 'Cassandra Brandow'
 'Resi P\x9alking' 'Carol Darley' 'Ralph Arnett' 'Naresj Patel'
 'Dorris liebe' 'Alejandro Savely' 'Jay Kimmel' 'David Philippe'
 'Seth Vernon' 'Kelly Collister' 'Christine Kargatis' 'Mathew Reese'
 'Emily Grady' 'Darrin Sayre' 'Maya Herman' 'Roland Fjeld' 'Melanie Seite'
 'Nona Balk' 'Anthony Johnson' 'Laurel Elliston' 'Benjamin Venier'
 'Bryan Mills' 'Liz Thompson' 'Sue Ann Reed' 'Carlos Daly'
 'Helen Wasserman' 'Gary McGarr' 'Allen Armold' 'Zuschuss Carroll'
 'Pauline Johnson' 'Brian Moss' 'Art Foster' 'Guy Armstrong' 'Greg Tran'
 'Anthony Jacobs' 'Chuck Clark' 'Harold Pawlan' 'Peter McVee'
```

Gambar 12 Kolom Customer Name

Berdasarkan hasil keluaran tersebut, sebagian besar format nama sudah terlihat normal (nama depan dan nama belakang). Namun, ditemukan anomali berupa karakter yang tidak terbaca, seperti pada nama "**Resi P\x9alking**". Karakter \x9a menunjukkan adanya kemungkinan masalah *encoding* atau kesalahan pengetikan dari sumber data. Hal ini menandakan bahwa kolom tersebut perlu melalui proses *data cleaning* untuk

memperbaiki karakter tidak valid agar nama pelanggan menjadi rapi dan valid.

h. Segment

```
print(df['Segment'].unique())
```

Perintah print(df['Segment'].unique()) berfungsi untuk mengidentifikasi dan menampilkan daftar lengkap tipe atau kelompok pelanggan yang terdapat di dalam kolom Segment.

```
print(df['Segment'].unique())

['Consumer' 'Corporate' 'Home Office']
```

Gambar 13 Kolom Segment

Berdasarkan hasil keluaran tersebut, diketahui bahwa target pasar penjualan terbagi menjadi tiga segmen pelanggan, yaitu Consumer, Corporate, dan Home Office. Temuan ini juga menunjukkan bahwa data pada kolom tersebut sudah bersih dan terstandarisasi, karena tidak ditemukan *inconsistent values* seperti kesalahan ketik atau variasi penulisan yang tidak disengaja.

i. Country

```
print(df['Country'].unique())
```

Perintah print(df['Country'].unique()) berfungsi untuk mengidentifikasi dan menampilkan daftar lengkap nilai unik atau nama negara yang terdapat di dalam kolom Country.

```
print(df['Country'].unique())

['United States']
```

Gambar 14 Kolom Country

Berdasarkan hasil keluaran tersebut, seluruh data transaksi pada *dataset* ini hanya berfokus pada satu negara, yaitu United States. Karena semua baris memiliki nilai yang sama dan tidak terdapat variasi penulisan, kolom ini

dapat dianggap sudah bersih. Namun, karena tidak memungkinkan perbandingan antarnegara, kolom ini berpotensi dihapus.

j. City

```
print(df['City'].unique())
```

Perintah print(df['City'].unique()) berfungsi untuk menampilkan daftar lengkap nilai unik atau nama-nama kota asal pelanggan yang terdapat di dalam kolom City.

```
print(df['City'].unique())

['Henderson' 'Fort Lauderdale' 'Los Angeles' 'Philadelphia' 'Orem'
 'Richardson' 'Houston' 'New Albany' 'Troy' 'Chicago' 'Memphis' 'Columbia'
 'Minneapolis' 'New York City' 'Aurora' 'Seattle' 'Wilmington'
 'Bloomington' 'Roseville' 'Newark' 'Franklin' 'San Antonio' 'Denver'
 'Medina' 'Dublin' 'Detroit' 'Tampa' 'Lakeville' 'San Francisco'
 'Columbus' 'Colorado Springs' 'Belleville' 'Arvada' 'Hackensack'
 'Murfreesboro' 'Manchester' 'Pembroke Pines' 'Lawrence' 'Jackson'
 'Canton' 'Akron' 'Concord' 'Decatur' 'Phoenix' 'Park Ridge' 'Amarillo'
 'Costa Mesa' 'Parker' 'Gladstone' 'Lakeland' 'Fort Worth' 'Jacksonville'
 'Richmond' 'San Diego' 'Dearborn' 'Warner Robins' 'Vallejo' 'Vancouver'
 'Arlington' 'Cleveland' 'Tyler' 'Waynesboro' 'Long Beach' 'Dover'
 'Oceanside' 'Trenton' 'Saint Petersburg' 'Des Moines' 'Lancaster'
 'Lake Elsinore' 'Omaha' 'Edmonds' 'Santa Ana' 'Florence' 'Louisville'
 'Lorain' 'Springfield' 'Rochester' 'Norwich' 'Milwaukee' 'Torrance'
 'Mesa' 'Round Rock' 'Auburn' 'Olympia' 'Washington' 'Jefferson City'
 'Saint Peters' 'Rockford' 'Miami' 'Yonkers' 'Monroe' 'Encinitas'
 'Jonesboro' 'Cuyahoga Falls' 'Harrisonburg' 'Rockville' 'Coral Springs'
 'Boynton Beach' 'Fresno' 'Chester' 'La Porte' 'Pasadena' 'Evanston'
 'Tucson' 'Pico Rivera' 'Provo' 'Smyrna' 'Middletown' 'Vineland'
 'Glendale' 'North Las Vegas' 'Allentown' 'Laguna Niguel' 'Everett'
 'Watertown' 'El Paso' 'Peoria' 'Carrollton' 'Kent' 'Lafayette' 'Oakland'
 'Suffolk' 'Madison' 'San Jose' 'Redmond' 'Muskogee' 'Charlotte'
 'Bowling Green' 'Spokane' 'Fairfield' 'Broomfield' 'Montgomery'
 'Brentwood' 'Providence' 'Pomona' 'Ontario' 'Moorhead' 'Redlands'
 'Sunnyvale' 'Lansing' 'Grand Prairie' 'York' 'College Station' 'Dallas'
 'Cincinnati' 'Laredo' 'San Angelo' 'Little Rock' 'Marion' 'Littleton'
 'Bangor' 'Southaven' 'Roswell' 'Sioux Falls' 'Sacramento' 'Huntsville'
 'Holyoke' 'Farmington' 'Passaic' 'Fayetteville' 'Mishawaka' 'Westfield'
 'Scottsdale' 'Malden' 'Bellingham' 'Alexandria' 'Wheeling' 'Raleigh'
 'Tulsa' 'Hesperia' 'Gilbert' 'Lowell' 'Hialeah' 'Lakewood' 'Bakersfield'
 'Pompano Beach' 'Corpus Christi' 'Redondo Beach' 'Las Cruces'
 'Chesapeake' 'Woodstock' 'Atlanta' 'Tempe' 'Lake Charles' 'Highland Park'
 'Saginaw' 'Skokie' 'Allen' 'Mount Pleasant' 'Murray' 'Pharr'
```

Gambar 15 Kolom City

Berdasarkan sebagian hasil keluaran tersebut, terlihat bahwa cakupan transaksi penjualan memiliki persebaran geografis yang luas di berbagai kota di United States, seperti Los Angeles, Philadelphia, dan Houston, serta banyak kota lainnya. Secara umum, penulisan nama kota sudah rapi dan konsisten menggunakan huruf kapital di awal kata, serta tidak menunjukkan anomali karakter rusak seperti yang sebelumnya ditemukan pada kolom Customer Name.

k. State

```
print(df['State'].unique())
```

Perintah print(df['State'].unique()) berfungsi untuk menampilkan daftar lengkap nilai unik atau nama-nama negara bagian tujuan pengiriman yang terdapat pada kolom State.

```
print(df['State'].unique())

['Kentucky' 'Florida' 'California' 'Pennsylvania' 'Utah' 'Texas' 'Indiana'
 'New York' 'Illinois' 'Tennessee' 'South Carolina' 'Minnesota' 'Colorado'
 'Washington' 'Delaware' 'Ohio' 'Wisconsin' 'Michigan' 'New Jersey'
 'Connecticut' 'Massachusetts' 'Mississippi' 'Arizona' 'Missouri'
 'Georgia' 'Virginia' 'Iowa' 'Nebraska' 'New Hampshire'
 'District of Columbia' 'North Carolina' 'Arkansas' 'Maryland' 'Alabama'
 'Nevada' 'Louisiana' 'Oregon' 'Oklahoma' 'Rhode Island' 'Maine'
 'South Dakota' 'New Mexico' 'Idaho' 'Vermont' 'Kansas' 'Wyoming'
 'Montana' 'West Virginia']
```

Gambar 16 Kolom State

Berdasarkan hasil keluaran tersebut, sebaran wilayah transaksi diketahui mencakup banyak negara bagian di United States, seperti California, Texas, dan New York. Serupa dengan temuan pada kolom *City*, penulisan nama negara bagian sudah rapi, terstandarisasi, dan tidak menunjukkan adanya *inconsistent values* maupun kesalahan ketik.

l. Postal Code

```
print(df['Postal Code'].unique())
```

Perintah print(df['Postal Code'].unique()) berfungsi untuk menampilkan daftar lengkap nilai unik dari kode pos wilayah pelanggan yang terdapat pada kolom Postal Code.

```
print(df['Postal Code'].unique())

[42420 33311 90032 19140 84057 75080 77041 47150 12180 90004 60610 38109
 77070 29203 55407 10009 80013 10035 98103 19805 61701 95661 43055 53132
 98115 78207 80219 37064 44256 43017 48227 38401 33614 55044 94109 60623
 43229 80906  7109 10024 80004  7601 37130 19143 94122  6040  2038 33024
 77036 94110 98105  1841 39212 48187 47201 44312 94521 62521 85023 60068
 79109 90045 92627 80134 64118 48234 33801 90036 76106 32216 40475 92024
 10011 48126 31088 94591 98661 60505 76017 44105 60653 75701 22980 90805
 19901 48205 11572 92105 48183 33710 22204 50315 93534 23223 92530 68104
 98026 92704 41042 40214 44052 22153  3301 14609  6360 53209 90503 85204
 19120 78664 92054 19134 80027 13021 92037 98502 20016 65109 63376 61107
 33142 77095 10701 28110 90049 72401 44221 22801 20852 33065 33437 93727
 19013 46350 31907 91104 90008 60201 85705 90660 84604 30080 35630  6457
  8360 77506 85301 89031 18103 92677  2149 13601 79907 61604 75007 98031
 70506 94601 23434 53711 95123 98002 97756 74403 28205 42104 89015 99207
 11561 45014 33178 97477 80020 36116 94513 65807 36830  2908 55901 85345
 47374 91767 91761 56560 92374 94086 48911  3820 75051 17403 77840 75081
 45231 78041 76903 72209 17602 43302 80122  4401 38671 30076 57103 95823
 77340  1040 87401  7055 28314 46544  7090 85254  2148 98226 47905 22304
 60090 27604 74133 92345 85234  1852 33012 44107 19711 93309 33068 78415
 90278 88001 23320 60098 30318 85281 70601 60035 48601 60076 75002 29464
 42071 78577 84084 78521 48146 97030 71203 37421 83642 75217 78745 32303
 80525  6824 39503  2169  6010 92691 89115 11550 60477 52001 43130 90301
  2151 48127 36608 60188 56301 75023 75220 49201 21044 31204 45503 33317
 20735 43615 75150 79762 53214 91911 27405 28540 37620 61821 78550 46142
 37211 90712  2895 53142 34952 33023 68025 11520 83704  2920 21215 61032
 38301  8701 77642 95610 86442 10801 91776 45011 30328 44060 73505 23666
 13440 95336 94526 48858  6708 14215 99301 30062 44134 77573 20877 73120
 27217 32725 92804 85224 63122 48180 48640 76706 71854 18018  5408 75043
 37918 67212 48237 32137 66062 84020 54601 33180  7960 92307 97301 83201
 32712  6460 94533 60089 92630 28027 55113 72032  6450 82001 47401 35601
```

Gambar 17 Kolom Postal Code

Berdasarkan hasil keluaran tersebut, ditemukan temuan penting terkait kualitas data, beberapa kode pos hanya terdiri dari 4 digit, seperti 1040, 2169, atau 7055. Standar ZIP Code di United States selalu berjumlah 5 digit. Anomali ini kemungkinan terjadi karena kolom terbaca sebagai tipe data *integer*, sehingga angka nol di depan otomatis terhapus. Oleh karena itu, diperlukan proses data cleaning dengan mengubah tipe data kolom menjadi *string* dan mengembalikan nol yang hilang agar data geografis kembali valid.

m. Region

```
print(df['Region'].unique())
```

Perintah print(df['Region'].unique()) berfungsi untuk mengidentifikasi dan menampilkan seluruh nilai unik atau pembagian wilayah operasional penjualan yang terdapat di dalam kolom Region.

```
print(df['Region'].unique())

['South' 'West' 'East' 'Central']
```

Gambar 18 Kolom Region

Berdasarkan hasil keluaran tersebut, target pasar terbagi ke dalam empat wilayah utama di United States, yaitu South, West, East, dan Central. Data pada kolom ini juga terlihat rapi, konsisten, dan tidak menunjukkan adanya *inconsistent values*.

n. Product ID

```
print(df['Product ID'].unique())
```

Perintah print(df['Product ID'].unique()) berfungsi untuk mengidentifikasi dan menampilkan daftar lengkap nilai unik atau kode identitas setiap produk yang terdapat pada kolom Product ID.

```
print(df['Product ID'].unique())

['FUR-BO-10001798' 'FUR-CH-10000454' 'FUR-TA-10000577' 'FUR-FU-10001487'
 'FUR-TA-10001539' 'FUR-CH-10002774' 'FUR-BO-10004834' 'FUR-FU-10004848'
 'FUR-FU-10003664' 'FUR-BO-10002545' 'FUR-CH-10004218' 'FUR-FU-10001706'
 'FUR-CH-10003061' 'FUR-CH-10003968' 'FUR-FU-10000397' 'FUR-CH-10001146'
 'FUR-CH-10000513' 'FUR-FU-10003708' 'FUR-FU-10000260' 'FUR-FU-10003194'
 'FUR-CH-10000863' 'FUR-FU-10003799' 'FUR-FU-10000629' 'FUR-FU-10004006'
 'FUR-FU-10001934' 'FUR-TA-10001768' 'FUR-FU-10002157' 'FUR-CH-10004063'
 'FUR-TA-10004534' 'FUR-FU-10003773' 'FUR-FU-10000521' 'FUR-CH-10004287'
 'FUR-CH-10003817' 'FUR-CH-10004698' 'FUR-BO-10002613' 'FUR-TA-10004289'
 'FUR-FU-10002960' 'FUR-BO-10004695' 'FUR-FU-10000087' 'FUR-TA-10003748'
 'FUR-CH-10004860' 'FUR-CH-10004477' 'FUR-TA-10001705' 'FUR-TA-10003473'
 'FUR-FU-10004017' 'FUR-FU-10000576' 'FUR-FU-10004351' 'FUR-TA-10002607'
 'FUR-FU-10002505' 'FUR-CH-10002965' 'FUR-FU-10003274' 'FUR-TA-10002903'
 'FUR-BO-10004709' 'FUR-FU-10004091' 'FUR-CH-10001891' 'FUR-FU-10002918'
 'FUR-FU-10001861' 'FUR-TA-10002533' 'FUR-FU-10002671' 'FUR-FU-10001918'
 'FUR-CH-10004086' 'FUR-FU-10001756' 'FUR-FU-10001588' 'FUR-CH-10002602'
 'FUR-FU-10001967' 'FUR-CH-10003312' 'FUR-TA-10004575' 'FUR-BO-10002268'
 'FUR-CH-10000785' 'FUR-FU-10000023' 'FUR-TA-10004915' 'FUR-TA-10000617'
 'FUR-FU-10001935' 'FUR-TA-10002041' 'FUR-BO-10001972' 'FUR-CH-10003956'
 'FUR-CH-10004886' 'FUR-FU-10004712' 'FUR-CH-10000015' 'FUR-FU-10003347'
 'FUR-CH-10003396' 'FUR-FU-10000246' 'FUR-CH-10002372' 'FUR-FU-10000206'
 'FUR-CH-10000225' 'FUR-CH-10002331' 'FUR-TA-10001889' 'FUR-BO-10002824'
 'FUR-BO-10001601' 'FUR-BO-10001337' 'FUR-TA-10001857' 'FUR-FU-10004864'
 'FUR-FU-10000073' 'FUR-CH-10001215' 'FUR-FU-10001290' 'FUR-CH-10003379'
 'FUR-CH-10000595' 'FUR-FU-10000448' 'FUR-CH-10002647' 'FUR-FU-10000723'
 'FUR-CH-10002024' 'FUR-FU-10003039' 'FUR-TA-10004256' 'FUR-FU-10004020'
 'FUR-CH-10004997' 'FUR-FU-10001475' 'FUR-FU-10001979' 'FUR-FU-10003849'
 'FUR-FU-10000010' 'FUR-CH-10004853' 'FUR-FU-10000221' 'FUR-CH-10001482'
 'FUR-FU-10000732' 'FUR-FU-10003096' 'FUR-FU-10002759' 'FUR-TA-10000688'
 'FUR-FU-10003553' 'FUR-BO-10004015' 'FUR-FU-10004071' 'FUR-FU-10003394'
 'FUR-TA-10002774' 'FUR-BO-10003272' 'FUR-FU-10002253' 'FUR-TA-10000198'
 'FUR-FU-10000222' 'FUR-TA-10002228' 'FUR-BO-10001619' 'FUR-FU-10003577'
```

Gambar 19 Kolom Product ID

Berdasarkan hasil keluaran tersebut, penulisan kode produk memiliki pola yang terstruktur dan seragam. Setiap kode diawali tiga huruf kapital), diikuti dua huruf kapital seperti “BO”, “CH”, atau “TA” yang mengindikasikan subkategori, lalu diakhiri delapan digit angka, seperti “FUR-BO-10001798”. Konsistensi format ini menunjukkan bahwa data pada kolom tersebut sudah bersih, tercatat dengan baik, dan tidak memiliki masalah kesalahan penulisan maupun *incosistent values*.

o. Category

```
print(df['Category'].unique())
```

Perintah print(df['Category'].unique()) berfungsi untuk menampilkan seluruh nilai unik atau daftar kategori produk yang ditawarkan di dalam kolom Category.

```
print(df['Category'].unique())

['Furniture']
```

Gambar 20 Kolom Category

Berdasarkan hasil keluaran tersebut, seluruh data transaksi pada *dataset* ini hanya berfokus pada satu kategori produk, yaitu Furniture. Sama seperti pada kolom *Country*, nilai pada kolom ini bersifat tunggal sehingga datanya sangat bersih dan konsisten. Namun, kondisi ini juga menunjukkan bahwa analisis perbandingan performa penjualan antar kategori produk utama tidak dapat dilakukan pada *dataset* ini.

p. Sub-Category

```
print(df['Sub-Category'].unique())
```

Perintah print(df['Sub-Category'].unique()) berfungsi untuk menampilkan daftar lengkap nilai unik atau pembagian spesifik jenis produk yang terdapat pada kolom Sub-Category.

```
print(df['Sub-Category'].unique())

['Bookcases' 'Chairs' 'Tables' 'Furnishings']
```

Gambar 21 Kolom Sub-Category

Berdasarkan hasil keluaran tersebut, produk turunan dari kategori utama Furniture terbagi menjadi empat subkategori, yaitu Bookcases, Chairs, Tables, dan Furnishings. Data pada kolom ini terlihat rapi, terstandarisasi dengan baik, serta tidak menunjukkan adanya *inconsistent values* atau kesalahan penulisan.

q. Product Name

```
print(df['Product Name'].unique())
```

Perintah print(df['Product Name'].unique()) berfungsi untuk menampilkan daftar lengkap nilai unik atau nama-nama spesifik dari produk yang terjual pada kolom Product Name.

```
print(df['Product Name'].unique())

['Bush Somerset Collection Bookcase'
 'Hon Deluxe Fabric Upholstered Stacking Chairs, Rounded Back'
 'Bretford CR4500 Series Slim Rectangular Table'
 'Eldon Expressions Wood and Plastic Desk Accessories, Cherry Wood'
 'Chromcraft Rectangular Conference Tables'
 'Global Deluxe Stacking Chair, Gray'
 'Riverside Palais Royal Lawyers Bookcase, Royale Cherry Finish'
 'Howard Miller 13-3/4" Diameter Brushed Chrome Round Wall Clock'
 "Electrix Architect's Clamp-On Swing Arm Lamp, Black"
 'Atlantic Metals Mobile 3-Shelf Bookcases, Custom Colors'
 "Global Fabric Manager's Chair, Dark Gray" 'Longer-Life Soft White Bulbs'
 'Global Leather Task Chair, Black' 'Novimex Turbo Task Chair'
 'Luxo Economy Swing Arm Lamp'
 "Global Value Mid-Back Manager's Chair, Gray"
 "High-Back Leather Manager's Chair"
 'Tenex Traditional Chairmats for Medium Pile Carpet, Standard Lip, 36" x 48"'
 '6" Cubicle Wall Clock, Black'
 'Eldon Expressions Desk Accessory, Wood Pencil Holder, Oak'
 'Novimex Swivel Fabric Task Chair' 'Seth Thomas 13 1/2" Wall Clock'
 '9-3/4 Diameter Round Wall Clock'
 'Deflect-o DuraMat Lighweight, Studded, Beveled Mat for Low Pile Carpeting'
 'Magnifier Swing Arm Lamp' 'Hon Racetrack Conference Tables'
 'Artistic Insta-Plaque' "Global Deluxe High-Back Manager's Chair"
 'Bevis 44 x 96 Conference Tables' 'Global Task Chair, Black'
 'Eldon Cleatmat Plus Chair Mats for High Pile Carpets'
 'Seth Thomas 14" Putty-Colored Wall Clock' 'SAFCO Arco Folding Chair'
 'Global Value Steno Chair, Gray' 'Padded Folding Chairs, Black, 4/Carton'
 'Atlantic Metals Mobile 4-Shelf Bookcases, Custom Colors'
 'BoxOffice By Design Rectangular and Half-Moon Meeting Room Tables'
 'Eldon 200 Class Desk Accessories, Burgundy'
 "O'Sullivan 2-Door Barrister Bookcase in Odessa Pine"
```

Gambar 22 Kolom Product Name

Berdasarkan hasil keluaran tersebut, penamaan produk terlihat sangat bervariasi dan detail, sering kali mencakup informasi seperti merek, model, jenis, hingga warna (misalnya *"Bush Somerset Collection Bookcase"* atau *"Global Deluxe Stacking Chair, Gray"*). Secara umum, format penulisan nama produk tampak rapi, informatif, dan selaras dengan subkategori Furniture yang telah dianalisis sebelumnya, tanpa menunjukkan adanya anomali karakter yang janggal.

r. Sales

```
print(df['Sales'].unique())
```

Perintah print(df['Sales'].unique()) berfungsi untuk menampilkan deretan nilai unik dari nominal transaksi yang terdapat pada kolom Sales.

```
print(df['Sales'].unique())

[261.96   731.94   957.5775 ... 383.4656  13.4     25.248 ]
```

Gambar 23 Kolom Sales

Berdasarkan hasil keluaran tersebut, terlihat deretan angka desimal (*float*) yang terpotong oleh tanda elipsis (...). Hal ini menunjukkan bahwa kolom tersebut berisi data numerik kontinu dengan variasi nilai yang sangat banyak sehingga tidak dapat ditampilkan seluruhnya. Meskipun demikian, hasil ini mengonfirmasi bahwa data sudah terekam dengan baik sebagai angka dengan presisi hingga beberapa tempat desimal (misalnya 957.5775). Namun, pengecekan nilai unik kurang ideal dan kurang informatif untuk menganalisis kolom dengan tipe data kontinu seperti ini.

s. Quantity

```
print(df['Quantity'].unique())
```

Perintah print(df['Quantity'].unique()) berfungsi untuk menampilkan seluruh nilai unik atau variasi jumlah barang yang dibeli per transaksi pada kolom Quantity.

```
print(df['Quantity'].unique())

[ 2  3  5  7  9  1  4  8  6 14 13 10 11 12]
```

Gambar 24 Kolom Quantity

Berdasarkan hasil keluaran tersebut, diketahui bahwa jumlah unit produk yang terjual berada pada rentang 1 hingga 14 unit. Berbeda dengan kolom *Sales* yang bersifat kontinu, data pada kolom ini merupakan numerik diskrit (bilangan bulat) dan kondisinya sangat bersih, karena tidak ditemukan nilai nol, angka negatif, maupun nilai ekstrem yang tidak logis.

t. Discount

```
print(df['Discount'].unique())
```

Perintah print(df['Discount'].unique()) berfungsi untuk menampilkan seluruh variasi nilai diskon yang diberikan pada setiap transaksi di dalam kolom Discount.

```
print(df['Discount'].unique())

[0.   0.45 0.2  0.3  0.5  0.6  0.32 0.1  0.15 0.4  0.7 ]
```

Gambar 25 Kolom Discount

Berdasarkan hasil keluaran tersebut, diketahui bahwa nilai diskon direkam dalam bentuk pecahan desimal (*float*) dengan rentang 0.0 hingga 0.7. Data pada kolom ini terlihat bersih dan logis, karena tidak ditemukan anomali seperti nilai diskon negatif maupun nilai di atas 1.0.

u. Profit

```
print(df['Profit'].unique())
```

Perintah print(df['Profit'].unique()) berfungsi untuk menampilkan deretan nilai unik dari keuntungan atau kerugian yang dihasilkan pada setiap transaksi di dalam kolom Profit.

```
print(df['Profit'].unique())

[  41.9136  219.582  -383.031  ...   10.2588    6.432     4.1028]
```

Gambar 26 Kolom Profit

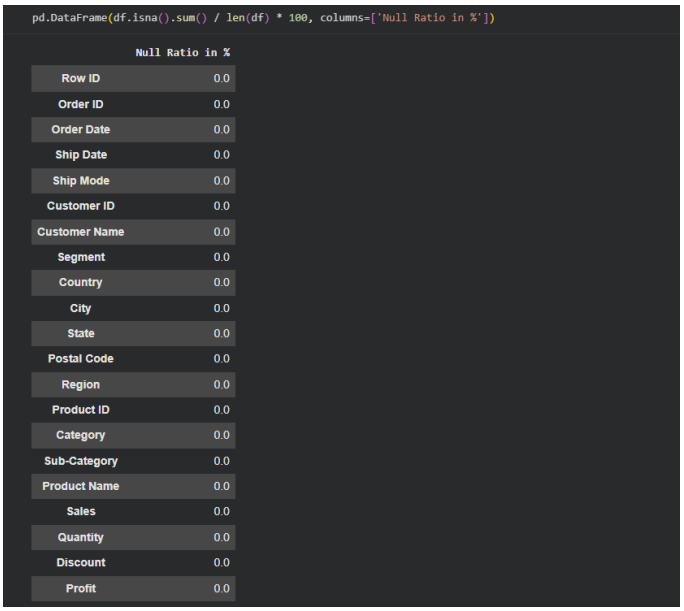
Berdasarkan hasil keluaran tersebut, terlihat variasi angka desimal (*float*) yang mencakup nilai positif dan nilai negatif. Sama seperti pada kolom *Sales*, data ini bersifat kontinu dengan variasi nilai yang sangat banyak sehingga tampilannya terpotong oleh tanda elipsis (...). Tetapi hasil ini mengonfirmasi bahwa data telah terekam dengan baik sebagai angka, namun pengecekan nilai unik kurang efektif untuk menggambarkan distribusi keseluruhan pada data numerik kontinu.

7. Missing Values

Missing Values adalah kondisi ketika terdapat data yang tidak terisi atau nilai yang hilang pada suatu kolom dalam *dataset*. Hal ini dapat terjadi karena berbagai alasan, seperti kesalahan saat proses pengumpulan data, data yang tidak dicatat, atau informasi yang memang tidak tersedia. Keberadaan *missing values* dapat mempengaruhi hasil analisis data, sehingga perlu dilakukan penanganan terlebih dahulu, misalnya dengan menghapus data tersebut atau mengisi nilai yang hilang dengan metode tertentu. Berikut penerapan *Missing Values* yang dapat dilakukan dengan perintah:

```
pd.DataFrame(df.isna().sum() / len(df) * 100,
columns=['Null Ratio in %'])
```

Perintah pd.DataFrame(df.isna().sum() / len(df) * 100, columns=['Null Ratio in %']) digunakan untuk menghitung dan menampilkan persentase *missing values* pada setiap kolom dalam bentuk tabel.



```
pd.DataFrame(df.isna().sum() / len(df) * 100, columns=['Null Ratio in %'])
```

	Null Ratio in %
Row ID	0.0
Order ID	0.0
Order Date	0.0
Ship Date	0.0
Ship Mode	0.0
Customer ID	0.0
Customer Name	0.0
Segment	0.0
Country	0.0
City	0.0
State	0.0
Postal Code	0.0
Region	0.0
Product ID	0.0
Category	0.0
Sub-Category	0.0
Product Name	0.0
Sales	0.0
Quantity	0.0
Discount	0.0
Profit	0.0

Gambar 27 Missing Values

Hasil eksekusinya menunjukkan nilai 0.0 pada seluruh kolom, yang mengonfirmasi bahwa dataset tersebut tidak memiliki data kosong dan bersih dari *incosistent values*.

8. Duplicated Values

Duplicated values adalah kondisi di mana terdapat dua baris atau lebih dalam sebuah *dataset* yang memiliki nilai sama persis di seluruh kolomnya, yang wajib dibersihkan karena dapat mengacaukan hasil perhitungan statistik akibat data yang terekam berulang. Berikut penerapan *Duplicated Values* yang dapat dilakukan dengan perintah:

```
df[df.duplicated()]
```

Perintah df[df.duplicated()] berfungsi untuk memfilter dan menampilkan baris-baris data yang merupakan duplikat.

Gambar 28 Duplicated Values

Hasil eksekusinya berupa tabel kosong dengan keterangan *0 rows*, yang artinya bahwa *dataset* tersebut sudah bersih dan tidak memiliki *Duplicated Values* sama sekali.

9. Outliers

Outliers adalah titik data yang nilainya sangat ekstrem atau menyimpang jauh dari mayoritas kelompok data lainnya dalam sebuah *dataset*, yang perlu mendapat perhatian khusus karena kehadirannya dapat mendistorsi hasil perhitungan statistik (seperti nilai rata-rata) menjadi tidak akurat. Berikut penerapan *Outliers* yang dapat dilakukan dengan perintah:

```
results = []


cols = df.select_dtypes(include=['float64', 'int64'])
```

```
for col in cols:
  q1 = df[col].quantile(0.25)
  q3 = df[col].quantile(0.75)
  iqr = q3 - q1
  lower_bound = q1 - 1.5*iqr
  upper_bound = q3 + 1.5*iqr
  outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
  percent_outliers = (len(outliers)/len(df))*100
  results.append({'Kolom': col, 'Persentase Outliers': percent_outliers})

# Dataframe dari list hasil
results_df = pd.DataFrame(results)
results_df.set_index('Kolom', inplace=True)
results_df = results_df.rename_axis(None, axis=0).rename_axis('Kolom',
axis=1)

# Tampilkan dataframe
display(results_df)
```

Perintah tersebut berfungsi untuk mendeteksi dan menghitung persentase pencilan (*outliers*) pada setiap kolom numerik menggunakan metode *Interquartile Range* (IQR).

```python
results = []

cols = df.select_dtypes(include=['float64', 'int64'])

for col in cols:
  q1 = df[col].quantile(0.25)
  q3 = df[col].quantile(0.75)
  iqr = q3 - q1
  lower_bound = q1 - 1.5*iqr
  upper_bound = q3 + 1.5*iqr
  outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
  percent_outliers = (len(outliers)/len(df))*100
  results.append({'Kolom': col, 'Persentase Outliers': percent_outliers})

# Dataframe dari list hasil
results_df = pd.DataFrame(results)
results_df.set_index('Kolom', inplace=True)
results_df = results_df.rename_axis(None, axis=0).rename_axis('Kolom', axis=1)

# Tampilkan dataframe
display(results_df)
```

Kolom	Persentase Outliers
Row ID	0.000000
Postal Code	0.000000
Sales	7.732202
Quantity	1.838755
Discount	0.000000
Profit	19.189062

Gambar 29 Outliers Values

Hasil eksekusinya menunjukkan adanya nilai ekstrem yang cukup signifikan pada kolom *Profit* (19.18%) dan *Sales* (7.73%).

2.2.2 EDA

Dalam EDA, ada beberapa *library Python* yang diperlukan untuk membantu menampilkan grafik, yaitu:

```
import seaborn as sns
import matplotlib.pyplot as plt
```

Perintah import seaborn as sns dan import matplotlib.pyplot as plt digunakan untuk mengaktifkan visualisasi data utama dalam *Python*, di mana Seaborn berfungsi untuk membuat grafik statistik yang estetik sementara Matplotlib menyediakan kontrol tingkat dasar untuk mengatur elemen dan tata letak grafik tersebut.

a. **Comparison**

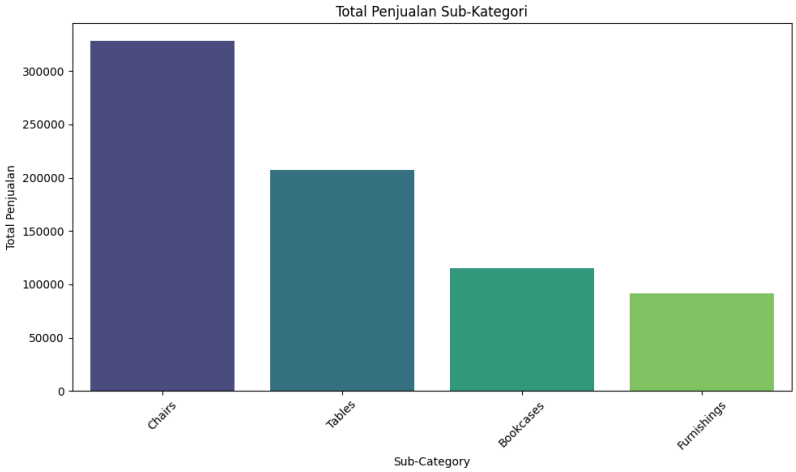
Comparison adalah proses mengevaluasi dua atau lebih sekumpulan data untuk mengidentifikasi persamaan, perbedaan, serta pola hubungan di antaranya guna mendapatkan wawasan yang lebih mendalam dalam pengambilan keputusan. Berikut penerapan Comparison yang dapat dilakukan dengan menggunakan barchart:

```
variabel = df.groupby('Sub-
Category')['Sales'].sum().sort_values(ascending=False)

plt.figure(figsize=(10,6))
sns.barplot(x=variabel.index, y=variabel.values, palette='viridis',
hue=variabel.index, legend=False)
plt.title('Total Penjualan Sub-Kategori')
plt.xlabel('Sub-Category')
plt.ylabel('Total Penjualan')
plt.xticks(rotation=45)
```

```
plt.tight_layout()
plt.show()
```

Untuk menerapkan *comparison* pada *bar chart*, digunakan perintah df.groupby('Sub-Category')['Sales'].sum() untuk mengelompokkan data berdasarkan sub-kategori lalu menjumlahkan seluruh nilai pada kolom *Sales* guna mengetahui total penjualan dari setiap kategori, yang kemudian diurutkan secara menurun menggunakan sort_values(ascending=False) dan divisualisasikan melalui sns.barplot() dengan penambahan judul, label sumbu, serta rotasi teks agar perbandingan performa penjualan antar sub-kategori pada grafik tersebut dapat terbaca secara lebih jelas dan sistematis.



Gambar 30 Comparison

Berdasarkan visualisasi terhadap sub-kategori produk, terlihat bahwa kategori Chairs menjadi sub-kategori dengan total penjualan tertinggi secara signifikan dibandingkan kategori lainnya. Sementara itu, kategori seperti Tables dan Bookcases berada pada tingkat penjualan menengah, sedangkan kategori Furnishings memiliki total penjualan yang lebih kecil. Hal ini menunjukkan bahwa konsumen memberikan kontribusi pendapatan terbesar melalui pembelian furnitur utama, khususnya kursi, yang memiliki volume penjualan paling dominan di pasar.

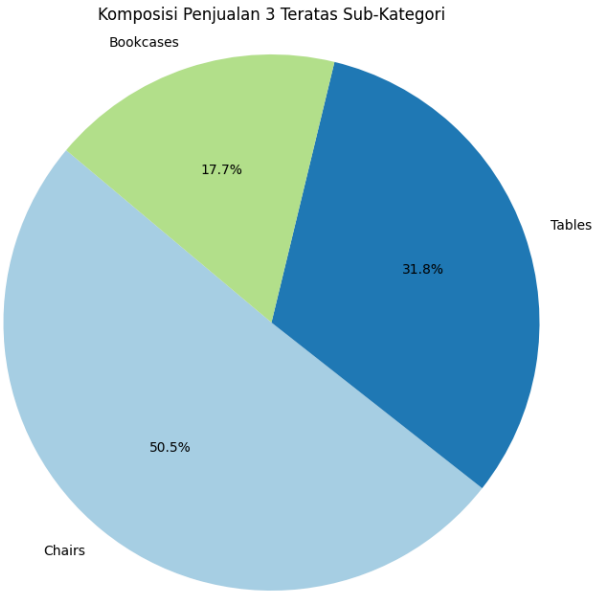
b. Composition

Composition adalah teknik visualisasi yang digunakan untuk menunjukkan bagaimana bagian-bagian individual membentuk satu kesatuan yang utuh, sehingga audiens dapat melihat kontribusi atau proporsi relatif dari setiap kategori terhadap total keseluruhan data. Berikut penerapan Composition yang dapat dilakukan dengan menggunakan piechart:

```
variabel = df.groupby('Sub-
Category')['Sales'].sum().sort_values(ascending=False)
variabel2 = variabel.head(3)

plt.figure(figsize=(10, 8))
variabel2.plot(kind='pie', autopct='%1.1f%%', startangle=140,
colors=plt.cm.Paired.colors)
plt.title('Komposisi Penjualan 3 Teratas Sub-Kategori')
plt.ylabel('')
plt.axis('equal')
plt.show()
```

Untuk menerapkan *composition* pada *pie chart*, digunakan perintah df.groupby('Sub-Category')['Sales'].sum() untuk menjumlahkan total penjualan tiap sub-kategori yang kemudian difilter menggunakan head(3) guna mengambil tiga kategori teratas, lalu divisualisasikan dengan plot(kind='pie') yang dilengkapi parameter autopct='%1.1f%%' untuk menampilkan persentase kontribusi masing-masing bagian terhadap total keseluruhan secara proporsional.



Gambar 31 Composition

Berdasarkan visualisasi terhadap tiga sub-kategori dengan penjualan tertinggi, terlihat bahwa kategori Chairs mendominasi komposisi secara masif dengan kontribusi sebesar 50.5%, disusul oleh Tables sebesar 31.8% dan Bookcases sebesar 17.7%, yang menunjukkan bahwa produk kursi merupakan kontributor pendapatan utama dibandingkan kategori furnitur lainnya dalam kelompok ini.

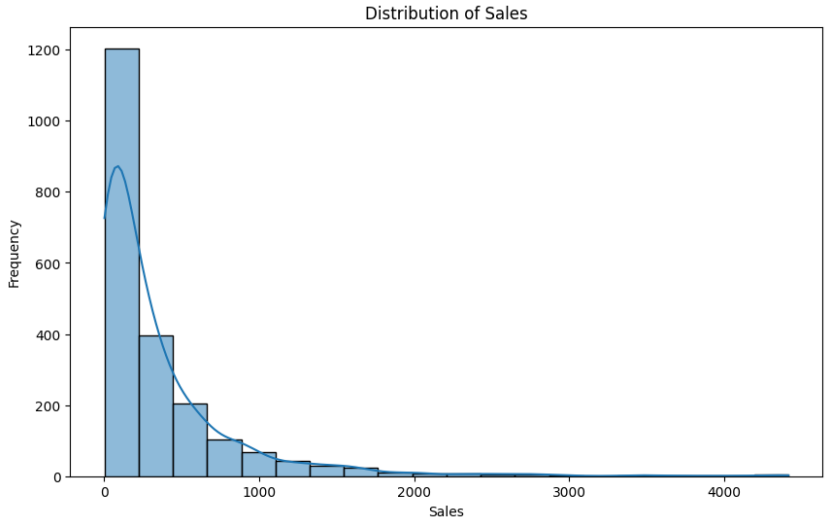
c. Distribution

Distribution adalah konsep statistik yang digunakan untuk menunjukkan bagaimana nilai-nilai dalam sebuah *dataset dataset* tersebar, seberapa sering nilai tersebut muncul, serta di mana konsentrasi data berada guna mengidentifikasi pola, rentang, hingga *outliers* dalam suatu variabel. Berikut penerapan Distribution yang dapat dilakukan dengan menggunakan histogram:

```
plt.figure(figsize=(10, 6))
sns.histplot(df['Sales'], bins=20, kde=True)
```

```
plt.title('Distribution of Sales')
plt.xlabel('Sales')
plt.ylabel('Frequency')
plt.show()
```

Untuk menerapkan *distribution* pada *histogram*, digunakan perintah sns.histplot(df['Sales']) untuk memvisualisasikan sebaran nilai pada kolom *Sales* dengan pembagian 20 *bins* serta penambahan kurva *Kernel Density Estimate* (KDE) buat melihat pola kepadatan data, di mana sumbu x menunjukkan nilai penjualan dan sumbu y menunjukkan frekuensi kemunculannya, lengkap dengan judul dan label sumbu agar informasi sebaran data tersebut mudah dipahami.



Gambar 32 Distribution

Berdasarkan visualisasi terhadap sebaran nilai penjualan, terlihat bahwa grafik memiliki bentuk *right-skewed* atau condong ke kanan, di mana mayoritas transaksi terkonsentrasi pada rentang nilai yang rendah (dibawah 500) dengan frekuensi tertinggi mencapai lebih dari 1200 kejadian. Sementara itu, jumlah transaksi terus menurun drastis seiring bertambahnya nilai penjualan, yang menunjukkan bahwa meskipun terdapat beberapa transaksi bernilai tinggi hingga di atas 4000 sebagai pencilan, sebagian besar pendapatan perusahaan berasal dari akumulasi transaksi dalam skala kecil hingga menengah.

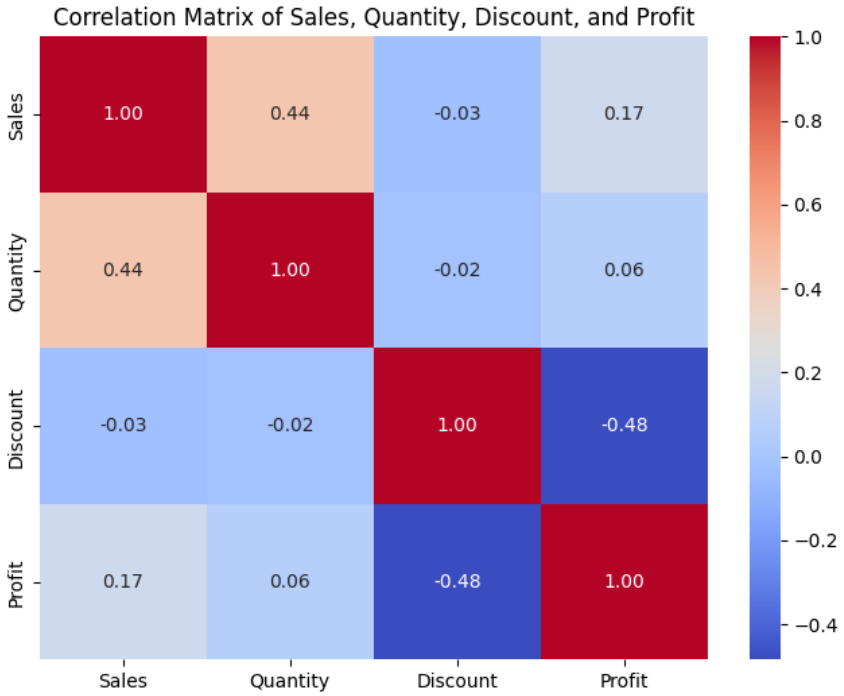
d. Relationship

Relationship adalah teknik analisis yang digunakan untuk melihat keterkaitan atau korelasi antara dua variabel atau lebih guna memahami bagaimana perubahan pada satu variabel dapat memengaruhi variabel lainnya, seperti apakah pemberian diskon besar secara langsung memengaruhi peningkatan laba atau justru sebaliknya. Berikut penerapan Relationship yang dapat dilakukan dengan menggunakan heatmap:

```
correlation_data = df[['Sales', 'Quantity', 'Discount', 'Profit']]

plt.figure(figsize=(8, 6))
sns.heatmap(data=correlation_data.corr(),
        annot=True,
        cmap='coolwarm',
        fmt='.2f')
plt.title('Correlation Matrix of Sales, Quantity, Discount, and Profit')
plt.show()
```

Untuk menerapkan *relationship* pada *heatmap*, digunakan perintah df[[‘Sales’, ‘Quantity’, ‘Discount’, ‘Profit’]].corr() untuk menghitung koefisien korelasi antar variabel numerik yang kemudian divisualisasikan melalui sns.heatmap() dengan parameter annot=True untuk menampilkan nilai korelasi dan cmap=’coolwarm’ untuk mengidentifikasi kekuatan serta arah hubungan antar variabel secara cepat melalui gradasi warna yang intuitif.



Gambar 33 Relationship

Berdasarkan visualisasi korelasi antar variabel numerik, terlihat bahwa terdapat hubungan negatif yang cukup kuat antara Discount dan Profit dengan nilai -0.48, yang menunjukkan bahwa semakin besar diskon yang diberikan, maka tingkat keuntungan cenderung akan menurun secara signifikan. Sementara itu, variabel Sales dan Quantity memiliki korelasi positif moderat sebesar 0.44, sedangkan hubungan antara variabel lainnya cenderung lemah atau mendekati nol, yang mengindikasikan bahwa volume penjualan tidak selalu menjamin peningkatan profitabilitas secara linear dalam dataset ini.

2.3 Data Preparation

Data Preparation merupakan serangkaian tahapan penting yang meliputi proses pembersihan, transformasi, dan pengorganisasian data mentah. Seperti menangani *missing values*, memperbaiki anomali atau karakter tidak valid, serta menyesuaikan tipe data, agar dataset menjadi akurat, konsisten, dan siap untuk tahap eksplorasi maupun pemodelan selanjutnya. Tahapan yang dilakukan dalam proses ini dapat dijelaskan sebagai berikut:

2.3.1 Penanganan Tipe Data

```
df.info()
```

Perintah df.info() berfungsi untuk menampilkan ringkasan terstruktur mengenai dataset.

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2121 entries, 0 to 2120
Data columns (total 21 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Row ID         2121 non-null   int64
 1   Order ID       2121 non-null   object
 2   Order Date     2121 non-null   object
 3   Ship Date      2121 non-null   object
 4   Ship Mode      2121 non-null   object
 5   Customer ID    2121 non-null   object
 6   Customer Name  2121 non-null   object
 7   Segment        2121 non-null   object
 8   Country        2121 non-null   object
 9   City           2121 non-null   object
 10  State          2121 non-null   object
 11  Postal Code    2121 non-null   int64
 12  Region         2121 non-null   object
 13  Product ID     2121 non-null   object
 14  Category       2121 non-null   object
 15  Sub-Category   2121 non-null   object
 16  Product Name   2121 non-null   object
 17  Sales          2121 non-null   float64
 18  Quantity       2121 non-null   int64
 19  Discount       2121 non-null   float64
 20  Profit         2121 non-null   float64
dtypes: float64(3), int64(3), object(15)
memory usage: 348.1+ KB
```

Gambar 34 df.info() before

Terlihat bahwa banyak tipe data yang salah seperti, Order Date dan Ship Date yang seharusnya tipe data (*datetime*) bukan *object*. Untuk mengubah tipe data dengan cara melakukan perintah berikut:

```
df['Order Date'] = pd.to_datetime(df['Order Date'], format='mixed')
```

```
df['Ship Date'] = pd.to_datetime(df['Ship Date'], format='mixed')
```

Untuk melakukan penyesuaian tipe data pada tahap *data preparation*, digunakan perintah pd.to_datetime() untuk mengubah kolom Order Date dan Ship Date menjadi format waktu (*datetime*), serta perintah astype(str) untuk mengonversi kolom Postal Code menjadi format teks (*string*).

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2121 entries, 0 to 2120
Data columns (total 21 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Row ID         2121 non-null   int64
 1   Order ID       2121 non-null   object
 2   Order Date     2121 non-null   datetime64[ns]
 3   Ship Date      2121 non-null   datetime64[ns]
 4   Ship Mode      2121 non-null   object
 5   Customer ID    2121 non-null   object
 6   Customer Name  2121 non-null   object
 7   Segment        2121 non-null   object
 8   Country        2121 non-null   object
 9   City           2121 non-null   object
 10  State          2121 non-null   object
 11  Postal Code    2121 non-null   int64
 12  Region         2121 non-null   object
 13  Product ID     2121 non-null   object
 14  Category       2121 non-null   object
 15  Sub-Category   2121 non-null   object
 16  Product Name   2121 non-null   object
 17  Sales          2121 non-null   float64
 18  Quantity       2121 non-null   int64
 19  Discount       2121 non-null   float64
 20  Profit         2121 non-null   float64
dtypes: datetime64[ns](2), float64(3), int64(3), object(13)
memory usage: 348.1+ KB
```

Gambar 35 df.info() after

Hasil validasi menggunakan df.info() menunjukkan bahwa seluruh kolom tersebut telah berhasil diperbarui ke format idealnya agar dapat diproses lebih lanjut tanpa memicu kesalahan kalkulasi matematis.

2.3.2 Incosistent Values

Inconsistent Values adalah kondisi di mana data yang merujuk pada informasi atau makna yang sama ditulis dengan format, ejaan, atau standar yang berbeda di dalam sebuah *dataset*.

a. Customer Name

```
# Menghapus angka dan simbol dari nama pelanggan agar konsisten
df['Customer Name'] = df['Customer Name'].str.replace(r'[^a-zA-Z\s]', '', regex=True)
```

Pada tahap ini, saya melakukan pembersihan karakter non-alfabet seperti angka dan simbol pada kolom Customer Name dengan tujuan menormalkan data identitas pelanggan, sehingga analisis frekuensi belanja per individu dapat menghasilkan data yang valid dan tidak terpecah akibat adanya karakter yang tidak relevan.

b. Product Name

```
# Menghapus tanda petik yang tidak konsisten di awal atau akhir string
df['Product Name'] = df['Product Name'].str.strip("'").str.strip('"')
```

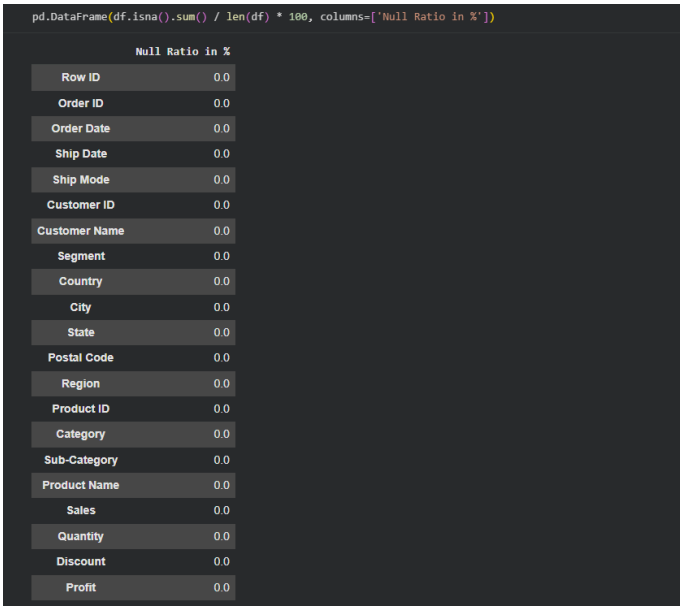
Pada tahap ini, tanda petik yang tidak berpasangan pada kolom Product Name dibersihkan dengan tujuan menyeragamkan penulisan nama produk, sehingga sistem dapat mengidentifikasi barang secara konsisten dan menghindari duplikasi entitas akibat perbedaan karakter penulisan.

2.3.3 Missing Values

Missing Values adalah kondisi ketika terdapat data yang tidak terisi atau nilai yang hilang pada suatu kolom dalam *dataset*. Hal ini dapat terjadi karena berbagai alasan, seperti kesalahan saat proses pengumpulan data, data yang tidak dicatat, atau informasi yang memang tidak tersedia. Keberadaan *missing values* dapat mempengaruhi hasil analisis data, sehingga perlu dilakukan penanganan terlebih dahulu, misalnya dengan menghapus data tersebut atau mengisi nilai yang hilang dengan metode tertentu.

```
pd.DataFrame(df.isna().sum() / len(df) * 100, columns=['Null Ratio in %'])
```

Perintah pd.DataFrame(df.isna().sum() / len(df) * 100, columns=['Null Ratio in %']) digunakan untuk menghitung dan menampilkan persentase *missing values* pada setiap kolom dalam bentuk tabel.



```
pd.DataFrame(df.isna().sum() / len(df) * 100, columns=['Null Ratio in %'])
```

	Null Ratio in %
Row ID	0.0
Order ID	0.0
Order Date	0.0
Ship Date	0.0
Ship Mode	0.0
Customer ID	0.0
Customer Name	0.0
Segment	0.0
Country	0.0
City	0.0
State	0.0
Postal Code	0.0
Region	0.0
Product ID	0.0
Category	0.0
Sub-Category	0.0
Product Name	0.0
Sales	0.0
Quantity	0.0
Discount	0.0
Profit	0.0

Gambar 36 Missing Values

Pada tahap ini, hasil pemeriksaan rasio data kosong menunjukkan nilai 0% pada seluruh 21 kolom, sehingga dataset bisa di bilang bersih dari missing values dan dapat langsung digunakan untuk proses lanjutan tanpa perlu menerapkan metode imputasi seperti pengisian nilai rata-rata atau median.

2.3.4 Duplicated Values

```
df = df.drop_duplicates()
```

Pada tahap ini, digunakan perintah df = df.drop_duplicates() untuk mengidentifikasi dan menghapus seluruh baris data yang terduplikasi, sehingga memastikan setiap catatan transaksi hanya terhitung satu kali guna mencegah terjadinya bias atau perhitungan ganda (*double counting*) pada proses analisis maupun visualisasi lanjutan.

2.3.5 Outliers

Outliers adalah titik data yang nilainya sangat ekstrem atau menyimpang jauh dari mayoritas kelompok data lainnya dalam sebuah *dataset*, yang perlu mendapat perhatian khusus karena kehadirannya dapat mendistorsi hasil perhitungan statistik (seperti nilai rata-rata) menjadi tidak akurat.

a. Profit

```
# Menghitung batas IQR untuk kolom Profit
Q1_profit = df['Profit'].quantile(0.25)
Q3_profit = df['Profit'].quantile(0.75)
IQR_profit = Q3_profit - Q1_profit

lower_limit = Q1_profit - 1.5 * IQR_profit
upper_limit = Q3_profit + 1.5 * IQR_profit

# Menerapkan Clipping (mengganti nilai di luar batas
dengan nilai batas tersebut)
df['Profit'] = df['Profit'].clip(lower=lower_limit,
upper=upper_limit)

# Ulangi hal yang sama untuk kolom Sales jika
diperlukan
```

Pada tahap ini, penanganan pencilan pada kolom Profit dan Sales dilakukan dengan metode clipping untuk mengurangi pengaruh nilai ekstrem tanpa menghilangkan baris data, sehingga kestabilan statistik dataset tetap terjaga dan model prediksi tidak terpengaruh oleh fluktuasi transaksi yang tidak wajar.

b. Sales

```
# Menghitung batas IQR untuk kolom Sales
Q1_sales = df['Sales'].quantile(0.25)
Q3_sales = df['Sales'].quantile(0.75)
IQR_sales = Q3_sales - Q1_sales

lower_s = Q1_sales - 1.5 * IQR_sales
upper_s = Q3_sales + 1.5 * IQR_sales

# Menerapkan Clipping pada Sales
df['Sales'] = df['Sales'].clip(lower=lower_s,
upper=upper_s)
```

Pada tahap ini, normalisasi pada kolom Sales dilakukan menggunakan metode clipping untuk menekan lonjakan transaksi yang bersifat anomali, sehingga model peramalan mampu menangkap pola penjualan yang lebih stabil dan menghasilkan prediksi pendapatan yang lebih realistis bagi perusahaan.

c. Quantity

```
# Menghitung batas IQR untuk kolom Quantity
Q1_qty = df['Quantity'].quantile(0.25)
Q3_qty = df['Quantity'].quantile(0.75)
IQR_qty = Q3_qty - Q1_qty

lower_q = Q1_qty - 1.5 * IQR_qty
upper_q = Q3_qty + 1.5 * IQR_qty
```

```
# Menerapkan Clipping pada Quantity
df['Quantity'] = df['Quantity'].clip(lower=lower_q,
upper=upper_q)
```

Pada tahap ini, penanganan pencilan pada kolom Quantity dilakukan dengan membatasi nilai pesanan yang terlalu ekstrem, sehingga model peramalan dapat lebih berfokus pada pola pembelian yang umum dan mewakili perilaku rata-rata konsumen di masa mendatang.

2.3.6 Construct Data

Tahap *construct data* adalah proses menciptakan atribut baru atau mentransformasikan variabel yang sudah ada dari dataset mentah guna menghasilkan informasi tambahan yang lebih bermakna dan relevan untuk meningkatkan kualitas analisis serta akurasi model prediktif.

a. Fitur Waktu (Tahun dan Bulan)

```
# Construct Data: Fitur Waktu (Tahun dan Bulan)
df['Order Year'] = df['Order Date'].dt.year
df['Order Month'] = df['Order Date'].dt.month
```

Langkah ini dilakukan dengan memecah kolom Order Date menjadi fitur Tahun dan Bulan agar model dapat mengenali pola musiman serta mempermudah analisis dan agregasi kinerja penjualan berdasarkan periode waktu tertentu.

b. Durasi Pengiriman (Processing Time)

```
# Construct Data: Durasi Pengiriman (Processing Time)
# Menghitung selisih hari antara pemesanan dan pengiriman
df['Processing Time (Days)'] = (df['Ship Date'] - df['Order
Date']).dt.days
```

Fitur ini dibentuk dengan menghitung selisih hari antara Order Date dan Ship Date sebagai Processing Time untuk merepresentasikan efisiensi

proses pengiriman dan memungkinkan analisis dampaknya terhadap kepuasan pelanggan maupun biaya operasional.

c. Harga Satuan (Price per Unit)

```
# Construct Data: Harga Satuan (Price per Unit)
# Mengetahui harga asli per barang sebelum dikali kuantitas
df['Price per Unit'] = df['Sales'] / df['Quantity']
```

Karena kolom Sales merepresentasikan total nilai transaksi, harga per unit dihitung dengan membagi Sales dengan Quantity agar nilai barang dapat dianalisis secara objektif tanpa dipengaruhi oleh jumlah pembelian dalam satu transaksi.

d. Profit Status

```
# Construct Data: Profit Status
def categorize_profit(profit):
    if profit > 0:
        return 'Untung'
    elif profit < 0:
        return 'Rugi'
    else:
        return 'Break Even'

df['Profit Status'] = df['Profit'].apply(categorize_profit)
```

Profit Status dibentuk melalui proses pelabelan kualitatif berdasarkan nilai pada kolom Profit dengan aturan sederhana, yaitu transaksi dikategorikan sebagai Untung jika profit bernilai positif, Rugi jika bernilai negatif, dan Break Even jika bernilai nol, sehingga data numerik yang abstrak dapat diubah menjadi kategori yang lebih mudah dipahami dan mempermudah pembuatan visualisasi untuk membandingkan transaksi yang sehat dan bermasalah.

```
# Menampilkan 5 baris dari kolom-kolom baru
print("\nSampel Kolom Baru:")
```

```
print(df[['Order Date', 'Order Year', 'Processing Time (Days)', 'Price per Unit', 'Profit Status']].head())
```

```
# Menampilkan 5 baris dari kolom-kolom baru
print("\nSampel Kolom Baru:")
print(df[['Order Date', 'Order Year', 'Processing Time (Days)', 'Price per Unit', 'Profit Status']].head())


Sampel Kolom Baru:
  Order Date  Order Year  Processing Time (Days)  Price per Unit Profit Status
0 2016-11-08        2016                       3        130.9800        Untung
1 2016-11-08        2016                       3        243.9800        Untung
2 2015-10-11        2015                       7        191.5155          Rugi
3 2014-06-09        2014                       5          6.9800        Untung
4 2014-06-09        2014                       5        113.0400        Untung
```

Gambar 37 Construct Data

Pada tahap ini, fitur baru seperti Order Year, Processing Time (Days), Price per Unit, dan Profit Status dibuat dengan mengolah kolom yang sudah ada, yaitu Order Date, Ship Date, Sales, Quantity, dan Profit. Langkah ini bertujuan agar data transaksi yang awalnya masih mentah menjadi lebih mudah dianalisis dan dapat digunakan untuk melihat pola penjualan serta kinerja pengiriman dengan lebih jelas.

2.3.7 Data Reduction

Data reduction merupakan proses optimalisasi untuk mengurangi volume atau dimensi *dataset* melalui teknik seperti agregasi, *feature selection*, atau penghapusan atribut yang tidak relevan, untuk meningkatkan efisiensi komputasi dan mempercepat analisis tanpa menghilangkan informasi penting dalam data.

```
# List kolom yang akan dihapus berdasarkan analisis relevansi
columns_to_drop = ['Row ID', 'Order ID', 'Customer ID', 'Customer Name', 'Country', 'Category']

# Melakukan Data Reduction
df = df.drop(columns_to_drop, axis=1)
```

```
# Verifikasi sisa kolom
print(df.columns)
```

Pada tahap ini, kolom yang bersifat redundan atau memiliki variansi nol, seperti Country dan Category, dihapus untuk merampingkan dataset. Langkah ini membantu mengurangi kompleksitas data, mempercepat proses pelatihan model, serta menghindari model mempelajari informasi yang tidak relevan dibandingkan pola penjualan yang sebenarnya.

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2121 entries, 0 to 2120
Data columns (total 20 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Order Date              2121 non-null   datetime64[ns]
 1   Ship Date               2121 non-null   datetime64[ns]
 2   Ship Mode               2121 non-null   object
 3   Segment                 2121 non-null   object
 4   City                    2121 non-null   object
 5   State                   2121 non-null   object
 6   Postal Code             2121 non-null   int64
 7   Region                  2121 non-null   object
 8   Product ID              2121 non-null   object
 9   Sub-Category            2121 non-null   object
 10  Product Name            2121 non-null   object
 11  Sales                   2121 non-null   float64
 12  Quantity                2121 non-null   float64
 13  Discount                2121 non-null   float64
 14  Profit                  2121 non-null   float64
 15  Order Year              2121 non-null   int32
 16  Order Month             2121 non-null   int32
 17  Processing Time (Days)  2121 non-null   int64
 18  Price per Unit          2121 non-null   float64
 19  Profit Status           2121 non-null   object
dtypes: datetime64[ns](2), float64(5), int32(2), int64(2), object(9)
memory usage: 315.0+ KB
```

Gambar 38 Data Reduction

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan hasil analisis data pada *Store Sales Forecasting Dataset*, dapat disimpulkan bahwa dataset memiliki kualitas yang cukup baik karena tidak ditemukan missing values maupun duplicated values. Namun, terdapat beberapa permasalahan seperti tipe data yang belum sesuai (kolom tanggal masih berupa string), adanya karakter tidak valid pada nama pelanggan, serta anomali pada kode pos dan outliers terutama pada kolom *Sales* dan *Profit*.

Dari hasil Exploratory Data Analysis (EDA), diketahui bahwa distribusi data penjualan cenderung tidak merata (skewed right) akibat adanya transaksi bernilai sangat tinggi. Selain itu, profit menunjukkan variasi yang sangat besar, bahkan terdapat transaksi dengan kerugian yang signifikan. Hal ini mengindikasikan bahwa strategi pemberian diskon memiliki pengaruh terhadap profit yang diperoleh.

Secara keseluruhan, analisis ini berhasil memberikan gambaran mengenai pola penjualan, kondisi data, serta faktor-faktor yang memengaruhi performa bisnis. Dashboard visualisasi yang dibuat juga mampu menyajikan informasi secara lebih mudah dipahami dan mendukung pengambilan keputusan.

3.2 Saran

Adapun saran yang dapat diberikan berdasarkan hasil analisis ini adalah sebagai berikut:

1. Meningkatkan efektivitas strategi pemberian diskon agar tidak menurunkan profit, khususnya pada transaksi yang berpotensi menyebabkan kerugian.

2. Mengoptimalkan penjualan pada sub-kategori produk yang memiliki kontribusi penjualan dan profit tinggi, serta melakukan evaluasi terhadap sub-kategori yang kurang menguntungkan.
3. Mengembangkan strategi pemasaran berdasarkan wilayah (region) untuk meningkatkan performa penjualan di setiap area secara lebih merata.
4. Melakukan perbaikan kualitas data, seperti konversi tipe data tanggal menjadi *datetime* dan perbaikan kesalahan encoding, agar analisis yang dihasilkan lebih akurat dan optimal.
5. Mengembangkan analisis lanjutan, seperti *forecasting* penjualan atau penambahan variabel analisis, guna memperoleh insight yang lebih mendalam dan mendukung pengambilan keputusan bisnis.